

PriVFim: Differentially Private Vertical Federated Frequent Itemset Mining

Xing Wang
Xi'an Jiaotong University
Xi'an, Shaanxi, China
xingwang@stu.xjtu.edu.cn

Jiasheng Yang
Xi'an Jiaotong University
Xi'an, Shaanxi, China
jiashengyang@stu.xjtu.edu.cn

Xinyu Li
Xi'an Jiaotong University
Xi'an, Shaanxi, China
2213311049@stu.xjtu.edu.cn

Xuebin Ren
Xi'an Jiaotong University
Xi'an, Shaanxi, China
xuebinren@mail.xjtu.edu.cn

Shusen Yang
Xi'an Jiaotong University
Xi'an, Shaanxi, China
shusenyang@mail.xjtu.edu.cn

ABSTRACT

Frequent itemset mining identifies groups of items that frequently occur together in user records. However, without a formal privacy guarantee, mining and releasing frequent itemsets may expose sensitive information about individual users. As a countermeasure, differential privacy (DP) is widely adopted to provide a rigorous privacy guarantee for frequent itemset mining by limiting the released information. Nevertheless, existing differentially private frequent itemset mining methods focus on non-federated or horizontal federated learning settings. Differentially private frequent itemset mining in vertical federated learning (VFL), where multiple data parties hold disjoint attributes of the same users, remains unexplored. In this paper, we investigate differentially private frequent itemset mining in VFL and propose PriVFim, a coarse-to-fine, two-round interactive framework for frequent itemset mining under VFL. In the first interaction, data parties report noisy local item counts, enabling the central server to construct an itemset-enumeration tree with a dynamic pruning strategy and select promising candidate itemsets. In the second interaction, the data parties report differentially private Flajolet-Martin sketches to reduce the estimation error, and the server applies a Maximum A Posteriori (MAP) method to refine cross-party support counts and select the global top- k itemsets. We provide a theoretical analysis of PriVFim and extensive experiments on real-world datasets demonstrate that PriVFim consistently outperforms baseline methods in mining accuracy while maintaining low computational and communication overhead.

PVLDB Reference Format:

Xing Wang, Jiasheng Yang, Xinyu Li, Xuebin Ren, and Shusen Yang.
PriVFim: Differentially Private Vertical Federated Frequent Itemset Mining.
PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://anonymous.4open.science/r/PrivFim-F2B0/>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Summary of representative DP studies under different settings where FIM refers to frequent itemset mining.

	Non-federated	HFL	VFL
FIM	[9, 18, 46, 60, 61]	[13, 16, 48, 70, 74]	Our work
Others	[20, 22, 24, 27, 31]	[4, 25, 32, 40, 62]	[35, 37, 71]

1 INTRODUCTION

Privacy regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA) impose increasingly strict requirements on the collection, sharing, and use of personal data [1, 2]. At the same time, many data analysis tasks require data held by different data parties to be used jointly, creating a conflict between data utility and privacy. *Federated learning* (FL) addresses this conflict by allowing data parties to jointly perform data analysis without sharing each party's raw user data [66]. FL is commonly divided into *horizontal federated learning* (HFL), where data parties hold the same attributes for different sets of users, and *vertical federated learning* (VFL), where data parties hold disjoint attributes for the same set of users. However, FL itself provides no formal privacy guarantee, and may leak users' private information. Secure Multi-Party Computation and encryption-based protocols have therefore been used to protect user data during federated computation [8, 52, 63, 73]. However, these methods do not provide protection for released results against membership inference [44, 49] or reconstruction attacks [10, 69], and they often incur substantial computation and communication overheads.

Recently, *differential privacy* (DP) has become a *de facto* standard for privacy protection because of its rigorous mathematical definition, low computational overhead, and resistance to membership inference and reconstruction attacks [23]. DP has been applied to diverse data analysis tasks, including frequency estimation [24, 58], mean/variance estimation [22, 57], range query [20, 53, 54, 65], streaming data [31, 47, 59] and graph data analysis [27–29]. Recent work has also incorporated DP into FL, with extensive studies in HFL [4, 25, 32, 40, 62] and a smaller number of studies in VFL considering tasks like clustering [37], tree boosting [35], and data synthesis [71]. In this work, we focus on frequent itemset mining, another important task under DP, where one central server aims to identify sets of items frequently appearing together in user records. Early DP studies on frequent itemset mining assumed that the central server had access to complete user records [9, 18, 26, 34, 51, 56, 67], while later methods adopted local

differential privacy, under which each user perturbs their record before reporting it to the server [17, 33, 46, 55, 60, 61]. However, these methods are designed for non-federated settings. Recent work has also extended frequent itemset mining [13, 16, 48, 70, 74] to federated settings. Nevertheless, these studies consider only the HFL setting, leaving differentially private frequent itemset mining under VFL unexplored. Table 1 summarizes current research status.

In this paper, we investigate differentially private frequent itemset mining in VFL. We aim to design an accurate and efficient framework that identifies the global top- k frequent itemsets, whose items may span multiple data parties, while ensuring that all reports and released results satisfy DP. Such a framework enables data parties to discover useful cross-attribute patterns that cannot be obtained from the local dataset of a single data party while protecting user privacy. For example, a bank and an e-commerce platform may use our framework to jointly discover associations between users’ financial profiles and purchasing behavior for risk assessment, without disclosing the raw user data held by either party. Achieving this framework involves three technical challenges:

- **Private and informative DP reporting.** In VFL, each data party has only part of a user’s record, while directly sharing raw user data would violate privacy requirements. It is challenging to design the reports of each party that satisfy DP while preserving sufficient information for global top- k frequent itemset mining.
- **Composable and accurate support count estimation.** After receiving the reports of each party, the central server should “compose” these reports to reconstruct correlations among attributes held by different data parties and thus to estimate the support counts of itemsets whose items span multiple data parties. Designing such a composable support count estimation method while achieving high estimation accuracy is challenging.
- **Efficient search over the global item universe.** Frequent itemset mining requires searching all subsets of the global item universe, whose size is exponentially large. It is challenging to design an efficient search strategy that reduces the mining cost while preserving the mining accuracy under DP.

To address these challenges, we propose PriVFim (Private Vertical Federated Itemset Mining), an efficient and accurate framework for differentially private top- k frequent itemset mining in VFL. PriVFim adopts a coarse-to-fine, two-round interaction architecture. In the first interaction, data parties report their noisy local item counts. Then, the central server constructs an *itemset-enumeration tree* and integrates a dynamic pruning strategy to rapidly shrink the exponentially large search space, filtering out unpromising itemsets and identifying a small set of candidate itemsets. In the second interaction, to reduce the estimation error, data parties encode the local user sets corresponding to these candidates using DP Flajolet-Martin (DP-FM) sketches and send these sketches to the server. Note that in the VFL setting, a user’s global record contains a cross-party itemset if and only if the user simultaneously exists in all corresponding local user sets. Thus, calculating the support count of an itemset is equivalent to estimating the intersection cardinality of these local user sets. While existing sketch-based methods calculate intersections via the inclusion-exclusion principle, they suffer from severe accuracy degradation. To overcome this, the second interaction of PriVFim estimates the intersection cardinalities using a *Maximum A Posteriori (MAP) method*, thereby

accurately refining the support counts of candidate itemsets to ultimately select the final top- k frequent itemsets. We provide a theoretical analysis of PriVFim on its privacy, utility, computational and communication costs. Extensive experiments demonstrate that our framework achieves precise top- k frequent itemset mining with relatively low computational and communication overhead. Our main contributions are summarized as follows:

- To the best of our knowledge, we are the first to investigate and propose a differentially private frequent itemset mining framework under the Vertical Federated Learning setting.
- We design PriVFim, a two-round interactive framework. It first utilizes an itemset-enumeration tree with a dynamic pruning strategy for efficient coarse screening. It then employs a MAP-based intersection cardinality estimation over multi-party DP-FM sketches for refined top- k frequent itemset selection.
- In addition to theoretical analysis, we implement PriVFim and conduct extensive experiments on real-world datasets. The experimental results demonstrate that our framework consistently outperforms baseline methods in mining accuracy while maintaining high computational and communication efficiency.

2 PRELIMINARIES

2.1 Differential Privacy

Differential privacy (DP) is a rigorous notion for privacy protection [23], which is defined as follows.

Definition 2.1 (Differential Privacy [23]). A randomized algorithm $\mathcal{A}$ satisfies (ϵ, δ) -DP if for any neighboring datasets D and D' that differ in one record and for every possible subset $\mathcal{O}$ of possible outputs of algorithm $\mathcal{A}$, $\Pr[\mathcal{A}(D) \in \mathcal{O}] \leq e^\epsilon \cdot \Pr[\mathcal{A}(D') \in \mathcal{O}] + \delta$.

DP has three properties that are frequently used to build complicated algorithms. Assume there are two algorithms $\mathcal{A}_1(\cdot)$ and $\mathcal{A}_2(\cdot)$ which can provide (ϵ_1, δ_1) -DP and (ϵ_2, δ_2) -DP, respectively. *Sequential composition* states that $\mathcal{A}_1(D, \mathcal{A}_2(D))$ satisfies $(\epsilon_1 + \epsilon_2, \delta_1 + \delta_2)$ -DP. *Parallel composition* demonstrates that combining two algorithms each only accessing a non-overlapping sub-dataset D_1 and D_2 satisfies $(\max\{\epsilon_1, \epsilon_2\}, \max\{\delta_1, \delta_2\})$ -DP. The third property, *post-processing* property, guarantees that any data-independent operation on an (ϵ, δ) -DP algorithm’s result still satisfies (ϵ, δ) -DP.

Laplace Mechanism. One of the most classic DP mechanisms is the Laplace mechanism. It adds Laplace noise to the output of a function f to ensure DP. Given a privacy budget ϵ , it is formalized as $\mathcal{A}_{\text{Lap}} = f(D) + \text{Lap}(\text{GS}_f/\epsilon)$, where $f(D)$ is the query result of the function f on dataset D . $\text{Lap}(\text{GS}_f/\epsilon)$ denotes a random variable sampled from the zero-mean Laplace distribution with scale GS_f/ϵ . GS_f is the L_1 -global sensitivity of f . It is calculated as $\text{GS}_f = \max_{D \approx D'} \|f(D) - f(D')\|_1$, where $D \approx D'$ means that datasets D and D' are neighboring (i.e., differ in one record).

2.2 DP Flajolet-Martin Sketch

The DP Flajolet-Martin (DP-FM) sketch is a probabilistic data structure designed for cardinality estimation on a large amount of data while protecting users’ privacy. Assume there are N users and each user u_i has a unique identifier $\text{id}_i \in [N] = \{1, \dots, N\}$. The DP-FM sketch aims to estimate the cardinality (i.e., the number of distinct identifiers) of a subset $X \subseteq [N]$. To satisfy DP, it injects *phantom*

Algorithm 1: DP Flajolet-Martin sketch

Input: Subset $\mathcal{X} \subseteq [N]$, privacy budget ϵ' , geometric distribution parameter γ , an ideal random hash function $H_\zeta(x) \sim \text{Geometric}(\frac{\gamma}{1+\gamma})$ with $\zeta \sim \text{Uniform}(\mathbb{Z})$

Output: Sanitized maximum hash value α

- 1 Calculate phantom elements bound $n_p = \lceil 1/(e^{\epsilon'} - 1) \rceil$;
- 2 Calculate lower bound $\alpha_{\min} = \lceil \log_{1+\gamma}(1/(1 - e^{-\epsilon'})) \rceil$;
- 3 Generate phantom element noise $\alpha_p = \max\{Y_1, \dots, Y_{n_p}\}$, where $Y_i \sim \text{Geometric}(\frac{\gamma}{1+\gamma})$;
- 4 Calculate the true maximum hash value $\alpha_{\text{real}} = \max\{H_\zeta(\text{id}_I) \mid \text{id}_I \in \mathcal{X}\}$;
- 5 **return** $\alpha = \max\{\alpha_{\min}, \alpha_p, \alpha_{\text{real}}\}$;

elements to mask small cardinalities and enforces a strict lower bound on the hash outputs. The DP-FM sketch is implemented with an ideal geometric-value hash function $H_\zeta : [N] \times \mathbb{Z} \rightarrow \mathbb{N}^+$, where $\zeta \sim \text{Uniform}(\mathbb{Z})$ is a hash key. Given any finite set of distinct elements in $[N]$, with the hash key ζ , the hashed values follow i.i.d. $\text{Geometric}(\frac{\gamma}{1+\gamma})$ distribution, where γ is the parameter of geometric distribution. Algorithm 1 provides the details of DP-FM sketch [50]. It first calculates the phantom elements bound n_p and the lower bound $\alpha_{\min}$ (Lines 1-2). It then draws n_p samples from the geometric distribution to obtain the phantom element noise α_p (Line 3). After calculating the true maximum hash value α_{real} from the actual elements in $\mathcal{X}$ (Line 4), it returns α which is the maximum of $\alpha_{\min}$, α_p , and α_{real} (Line 5). Finally, the cardinality of $\mathcal{X}$, i.e., $|\mathcal{X}|$, is estimated as $|\hat{\mathcal{X}}| = (1 + \gamma)^\alpha - n_p$. To ensure estimation accuracy, Algorithm 1 can be repeated h times using independent hash keys $\zeta_1, \dots, \zeta_h$, yielding a DP-FM sketch vector $\mathcal{M} = \{\alpha_1, \dots, \alpha_h\}$. Based on this, the cardinality of $\mathcal{X}$ can be estimated as follows:

$$|\hat{\mathcal{X}}| = (1 + \gamma)^{\text{HarmonicMean}(\mathcal{M})} - n_p, \quad (1)$$

where $\text{HarmonicMean}(\mathcal{M})$ calculates the harmonic mean of α_i in $\mathcal{M}$. The privacy guarantee of DP-FM sketch is given as follows:

LEMMA 2.2 (PRIVACY GUARANTEE OF DP-FM SKETCH [37]). *Given an ideal geometric-value hash function H_ζ , Algorithm 1 is ϵ' -DP (i.e., $(\epsilon', 0)$ -DP). And repeating it h times with different hash keys and $\epsilon' = \frac{\epsilon}{4\sqrt{h \log(1/\delta)}}$ satisfies (ϵ, δ) -DP, as long as $\epsilon \leq 2\log(1/\delta)$.*

Set Union Estimation. DP-FM sketch supports set union operations via element-wise maximums. Specifically, given two subsets $\mathcal{X}_1, \mathcal{X}_2 \subseteq [N]$ and their DP-FM sketch vectors $\mathcal{M}_1 = \{\alpha_{1,1}, \dots, \alpha_{1,h}\}$ and $\mathcal{M}_2 = \{\alpha_{2,1}, \dots, \alpha_{2,h}\}$, the sketch vector of their union $\mathcal{X}_1 \cup \mathcal{X}_2$ is derived as $\mathcal{M}_\cup = \{\max(\alpha_{1,1}, \alpha_{2,1}), \dots, \max(\alpha_{1,h}, \alpha_{2,h})\}$. The cardinality of the union, i.e., $|\mathcal{X}_1 \cup \mathcal{X}_2|$, is estimated from the merged sketch $\mathcal{M}_\cup$ using Eq. (1), calculated as $(1 + \gamma)^{\text{HarmonicMean}(\mathcal{M}_\cup)} - n_p$.

Set Intersection Estimation. Given two subsets $\mathcal{X}_1, \mathcal{X}_2 \subseteq [N]$, the existing approach [37] calculates the cardinality of the intersection $|\mathcal{X}_1 \cap \mathcal{X}_2|$ via the inclusion-exclusion principle, i.e., $|\mathcal{X}_1 \cap \mathcal{X}_2| = N - |\overline{\mathcal{X}_1} \cup \overline{\mathcal{X}_2}|$. To satisfy DP, N is estimated as $\hat{N} = N + \text{Lap}(1/\epsilon_0)$, which is the Laplace-noised total user count with privacy budget ϵ_0 . $\overline{\mathcal{X}_1} = [N] \setminus \mathcal{X}_1$ and $\overline{\mathcal{X}_2} = [N] \setminus \mathcal{X}_2$ represent the complementary sets of $\mathcal{X}_1$ and $\mathcal{X}_2$. To estimate $|\overline{\mathcal{X}_1} \cup \overline{\mathcal{X}_2}|$, the DP-FM sketches of $\overline{\mathcal{X}_1}$ and $\overline{\mathcal{X}_2}$ are generated via Algorithm 1 (Note that in real-world scenarios, the sketches of $\overline{\mathcal{X}_1}$ and $\overline{\mathcal{X}_2}$ may also be obtained by calculating the union over multiple DP-FM sketches [37] rather than directly from Algorithm 1). Then, the cardinality of their union $|\overline{\mathcal{X}_1} \cup \overline{\mathcal{X}_2}|$ is estimated using the aforementioned Set Union Estimation.

Table 2: Notations.

Notation	Description
K, N, M	Numbers of data parties, aligned users, and global attributes.
k	Number of global frequent itemsets to be returned.
U, u_i, id_i	Aligned user set, its i -th user, and the corresponding identifier.
A, A_i, a_i^j, m_i	Global attribute set; party i 's attribute set, its j -th attribute, and size.
D_i^j, s_i^j	Domain of attribute a_i^j and its number of categorical values.
$t = (a_i^j, v), I_i, I$	Item t at party i ; party i 's item universe and the global universe.
S_i^l, S^l	Local record of user u_i at party i and the corresponding global record.
I', C_P	An itemset and the number of global records containing it.
$\epsilon_1, \epsilon_2, \epsilon, \delta$	First/second interaction budgets, total budget, and DP parameter.
$c(t), \hat{c}(t), N_i, \bar{N}$	Exact/noisy count of t ; noisy user count from party i and its average.
F_i^j, F^j, P_i	Party i 's top- k set; its cross-party union; party i 's first-round report.
$\mathcal{T}_w, j_w, C_w$	Itemset-enumeration tree and the itemset/count at its node w .
σ_{2k}, C_{2k}	Dynamic pruning threshold and resulting top- $2k$ candidate set.
$\mathcal{F}_k$	Final global top- k frequent itemsets after refined estimation.
$J^{(r)}, C^{(r)}, \hat{C}^{(r)}, \tilde{C}^{(r)}$	The r -th candidate and its true, coarse, and refined support counts.
$J_i^{(r)}, \Omega_i, J_i^p$	Local part of $J^{(r)}$ and set of k selected itemsets at party i ; its p -th itemset in Ω_i .
$X_i^l, n_i^l, \hat{n}_i^l$	Local user (User-ID) set for J_i^p at party i ; its cardinality and MAP estimate.
$M_i^l, \alpha_i^{g,l}, h$	DP-FM sketch for X_i^l , its g -th output, and repetition count.
$\epsilon', \gamma, n_p, \alpha_{\min}$	DP-FM budget, geometric parameter, phantom count, and output lower bound.
$F(x)$	Cumulative distribution function of a single hashed value at x .
$\mathcal{Q}_i, \mathcal{B}^{(r)}$	Party i 's second-round report; matched (i, p) pairs for $J^{(r)}$.
$\tilde{L}^{(r)}, \tilde{U}^{(r)}, N^{(r)}$	Estimated bounds and feasible values of intersection count $C^{(r)}$.
$P_g^l(a \mid n), F_g^l(t \mid n)$	PMF and CDF of one output from M_i^l when $n_i^l = n$.
$\mathcal{L}_g^l(n), \mathcal{E}_{\text{disjoint}}$	Likelihood in repetition g given $C^{(r)} = n$; disjointness condition.

While the above enables set-interaction estimation, its standard deviation is $O((|\overline{\mathcal{X}_1} \cup \overline{\mathcal{X}_2}| + n_p)/\sqrt{h})$ [37]. In the scenario of frequent itemset mining, datasets are typically sparse, meaning that individual items are possessed by only a small fraction of users. Consequently, the complementary sets $\overline{\mathcal{X}_1}$ and $\overline{\mathcal{X}_2}$ contain the vast majority of users, causing their union $|\overline{\mathcal{X}_1} \cup \overline{\mathcal{X}_2}|$ to approach the total user count N . As a result, the standard deviation of the error scales with N , meaning it introduces massive noise. To resolve this, we propose another method for intersection estimation in Sec. 4.3.2.

3 PROBLEM FORMULATION

In this paper, we consider federated frequent itemset mining over vertically partitioned tabular datasets.

System Model. We consider a Vertical Federated Learning (VFL) scenario with K data parties and one untrusted central server, where each party holds data on different attributes of the same set of users. Specifically, we assume that all K parties share a unified user set $U = \{u_1, \dots, u_N\}$ of N users, where each user u_i is assigned a unique identifier $\text{id}_i \in [N] = \{1, \dots, N\}$. In VFL, the global attribute set is vertically partitioned among the K parties. Let $A = \{a_1, \dots, a_M\}$ denote the global attribute set containing M attributes in total. We assume each party $i \in [K]$ locally holds an attribute set $A_i = \{a_i^1, \dots, a_i^{m_i}\} \subseteq A$ of m_i attributes. Local attribute sets among K parties are mutually disjoint, i.e., $A_i \cap A_{i'} = \emptyset$ for $\forall i \neq i'$, and satisfy $\bigcup_{i=1}^K A_i = A$. For the j -th attribute held by party i , i.e., a_i^j , we denote its domain as $D_i^j = \{1, \dots, s_i^j\}$ containing s_i^j distinct categorical values. Following existing frequent itemset mining methods for tabular data [60], we define an *item* as an attribute-value pair (a_i^j, v) , where $i \in [K]$, $j \in [m_i]$, and $v \in D_i^j$. Thus, the universe of all possible items held by party i can be calculated as $I_i = \{(a_i^j, v) \mid \forall j \in [m_i], \forall v \in D_i^j\}$, and the global item universe across all K parties can be calculated as $I = \bigcup_{i=1}^K I_i$. For users' records, we denote user u_i 's local record at party i comprising m_i attribute values as $(v_i^{1,1}, \dots, v_i^{l, m_i})$. According to [60], this local record can also be represented as a set of items (i.e., a set of attribute-value pairs) $S_i^l = \{(a_i^j, v_i^{j,l}) \mid j \in [m_i]\}$ of size m_i . Thus, the global record representing all the attributes of user u_i across all parties can be calculated as $S^l = \bigcup_{i=1}^K S_i^l$. Based on this, for any itemset $I' \subseteq I$, its

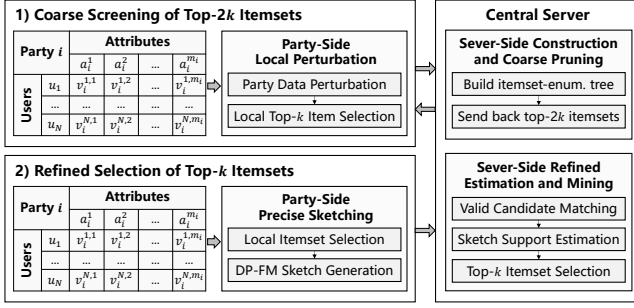


Figure 1: The overview of PriVFim.

support count $C_{I'}$ (i.e., the number of users whose global records contain I') is $C_{I'} = \sum_{l=1}^N \mathbb{I}(I' \subseteq S^l)$, where $\mathbb{I}(\cdot)$ is an indicator and outputs 1 if $I' \subseteq S^l$ and 0 otherwise. In VFL frequent itemset mining, the central server aims to coordinate all K parties to find top- k most frequent itemsets (i.e., k itemsets with the highest $C_{I'}$).

Adversary Model. We consider the central server to be semi-honest, i.e., an honest-but-curious adversary who will faithfully execute the collaborative mining protocol but may attempt to infer private individual records from the intermediate data received. Furthermore, we assume that none of the data parties will collude with the central server or with one another. We then consider that there may be third-party analysts or eavesdroppers who aim to infer sensitive private information about the input dataset by analyzing the final released Top- k frequent itemsets or the intermediate messages exchanged between the parties and the server.

Our Goal. Frequent itemset mining requires the central server to discover the top- k most frequent itemsets I' that yield the highest support counts $C_{I'}$. However, because global records S^l are vertically split across K parties, calculating the support counts requires information exchange between the central server and data parties, which may expose sensitive user information to the untrusted central server or eavesdroppers under our adversary model. To resolve this conflict between data utility and privacy, we aim to design an efficient VFL framework that accurately identifies the top- k frequent itemsets while guaranteeing that all messages exchanged with the central server and the final outputs strictly satisfy (ϵ, δ) -DP. Table 2 summarizes key notations used in this paper.

4 OUR SOLUTION

4.1 Overview

To achieve our goal of accurately mining global frequent itemsets under VFL while satisfying DP, we propose PriVFim (Private Vertical Federated Itemset Mining). To reduce both estimation error and computational complexity when searching over the global item universe I , inspired by [60], we adopt a coarse-to-fine, two-round interaction method between the data parties and the central server. The first interaction serves as a coarse screening phase, which reduces the search space by selecting top- $2k$ frequent itemsets. Building upon these $2k$ candidate itemsets, the second interaction employs a refined estimation mechanism using FM sketches, which achieve smaller errors for intersection estimation, to accurately calculate support counts and select the global top- k frequent itemsets. The overview of PriVFim is shown in Figure 1. Specifically, PriVFim consists of the following two interactive phases:

1) First Interaction: Coarse Screening of Top-2k Itemsets

- **Party-Side Local Perturbation.** Each data party $i \in [K]$ first calculates the local support counts for all items (i.e., attribute-value pairs as introduced in Sec. 3) in its local item universe I_i . To protect privacy, party i then perturbs the counts of items using the Laplace mechanism. To facilitate the estimation of support counts at the server side, party i also perturbs the total number of users. These perturbations overall consume ϵ_1 privacy budget. After that, party i selects its local top- k items based on the noisy counts of items and sends these top- k items together with their perturbed counts, as well as the perturbed number of total users, to the central server.

- **Server-Side Tree Construction and Coarse Pruning.** Upon receiving the local top- k items and counts from all K parties, the central server constructs an *itemset-enumeration tree*, where each node represents an itemset and its corresponding support count. While building the tree, the server applies a pruning strategy to prune nodes with small estimated counts. Through this process, the server selects $2k$ itemsets with the highest estimated counts as candidate itemsets and sends these $2k$ candidate itemsets, together with their estimated counts, back to the data parties.

2) Second Interaction: Refined Selection of Top-k Itemsets

- **Party-Side Precise Sketching.** Upon receiving the $2k$ candidate itemsets and their estimated counts from the first interaction, each data party selectively chooses the corresponding local frequent itemsets and reports their privatized FM sketches (i.e., DP-FM sketches introduced in Sec. 2.2) to the central server with a privacy budget ϵ_2 . Note that we use DP-FM sketches here to achieve smaller estimation errors when calculating intersection cardinalities.

- **Server-Side Refined Estimation and Mining.** Using the received DP-FM sketches, the central server estimates the exact support counts for the $2k$ candidate itemsets, finally selecting the top- k global frequent itemsets. Note that in this phase, the server adopts a *Maximum A Posteriori (MAP) method* for intersection estimation to reduce the estimation error under DP-FM sketches.

In what follows, we will detail the first and the second rounds of interaction in Sec. 4.2 and Sec. 4.3, respectively.

4.2 Coarse Screening of Top-2k Itemsets

4.2.1 Party-Side Local Perturbation. Party-Side Local Perturbation consists of two stages as follows.

Party Data Perturbation. In the first interaction, each data party $i \in [K]$ independently processes its local dataset to identify locally frequent items while enforcing DP guarantees. Specifically, for each local item $t \in I_i$, party i first calculates its exact local support count $c(t)$, defined as the total number of users whose local records contain item t , i.e., $c(t) = \sum_{l=1}^N \mathbb{I}(t \in S_i^l)$, where $\mathbb{I}(\cdot)$ is the indicator function. To satisfy DP, party i evenly divides a privacy budget $\epsilon_{1,1} = 0.9\epsilon_1$ across all $|I_i|$ local items, allocating a budget of $\epsilon_{1,1}/|I_i|$ to each item. Party i then perturbs each exact count $c(t)$ using the Laplace mechanism, calculating the noisy local support count as $\tilde{c}(t) = c(t) + \text{Lap}(|I_i|/\epsilon_{1,1})$. In addition to perturbing local support counts, to help estimate the support counts of itemsets at the server, party i also perturbs the total number of users N to $\tilde{N}_i = N + \text{Lap}(1/\epsilon_{1,2})$ with a privacy budget $\epsilon_{1,2} = 0.1\epsilon_1$.

Local Top- k Item Selection. Since globally frequent itemsets are predominantly composed of items that are individually frequent

at local parties, sending infrequent items would incur unnecessary communication cost and explode the search space for the central server. Therefore, party i ranks all items in I_i based on their noisy counts $\tilde{c}(t)$ and retains only the top- k most frequent items to form a local candidate item set $\mathcal{I}_i^c = \{I_i^{c,1}, \dots, I_i^{c,k}\} \subseteq I_i$ of size k . Finally, party i sends the selected top- k items along with their corresponding noisy counts, denoted as $\mathcal{P}_i = \{(I_i^{c,j}, \tilde{c}(I_i^{c,j})) \mid j \in [k]\}$, as well as the perturbed number of users $\hat{N}_i$, to the central server.

4.2.2 Server-Side Tree Construction and Coarse Pruning.

After receiving the data from each data party, the central server processes them and identifies top-2 k frequent itemsets as follows.

Tree Construction. Upon receiving the local items and their noisy counts $\mathcal{P}_i = \{(I_i^{c,j}, \tilde{c}(I_i^{c,j})) \mid j \in [k]\}$ from all K data parties, the central server aggregates the top- k items collected from all K parties into a set $\mathcal{I}^c = \bigcup_{i=1}^K \{I_i^{c,j} \mid j \in [k]\}$ of size $K \cdot k$. For the convenience of subsequent derivation, we denote $\mathcal{I}^c[r] \in \mathcal{I}^c$ for $r \in \{1, \dots, K \cdot k\}$ as the $r - (\lceil r/k \rceil - 1)k$ -th item sent by the $\lceil r/k \rceil$ -th data party, i.e., $\mathcal{I}^c[r] = I_{\lceil r/k \rceil}^{c, r - (\lceil r/k \rceil - 1)k}$. Under this definition, $\mathcal{I}^c[1], \dots, \mathcal{I}^c[k]$ represent the items sent by the first party, i.e., $I_1^{c,1}, \dots, I_1^{c,k}$, respectively, while $\mathcal{I}^c[k+1], \dots, \mathcal{I}^c[2k]$ represent the items sent by the second party, i.e., $I_2^{c,1}, \dots, I_2^{c,k}$, respectively, and so forth. For any item $\mathcal{I}^c[r]$ in $\mathcal{I}^c$, its corresponding noisy support count reported by the $\lceil r/k \rceil$ -th party is denoted as $\tilde{c}(\mathcal{I}^c[r])$.

Based on this, the central server constructs an *itemset-enumeration tree* $\mathcal{T}$ to enumerate all the itemsets that consist of the items in $\mathcal{I}^c$ and estimate their support counts, aiming to find the top-2 k most frequent itemsets for this interaction phase. In the tree $\mathcal{T}$, each node w represents an itemset $J_w \subseteq \mathcal{I}^c$ and its estimated support count $\tilde{C}_w$. The server constructs $\mathcal{T}$ layer by layer. At layer 0, the server initializes the root node w_{root} representing the empty itemset $J_{w_{\text{root}}} = \emptyset$. To construct layer 1, the server attaches $K \cdot k$ child nodes to the root, where the itemset J_{w_r} of each child node w_r , $r \in [K \cdot k]$, is set as $J_{w_r} = \{\mathcal{I}^c[r]\}$. And its estimated support count is set directly using the received noisy count, i.e., $\tilde{C}_{w_r} = \tilde{c}(\mathcal{I}^c[r])$. This enumerates all itemsets of size 1 formed by items in $\mathcal{I}^c$. Proceeding iteratively from layer 1, for each layer $d = 1, 2, \dots$, the server expands the nodes representing itemsets of size d in layer d to construct child nodes representing itemsets of size $d+1$ in layer $d+1$. Specifically, for any node w at layer d with an itemset $J_w = \{\mathcal{I}^c[r_1], \mathcal{I}^c[r_2], \dots, \mathcal{I}^c[r_d]\}$ where $r_1 < r_2 < \dots < r_d$, the server determines valid items to extend J_w and generates its child nodes. To prevent duplicate search and eliminate invalid attribute combinations, an item $\mathcal{I}^c[q]$ (where $q \in [K \cdot k]$) is valid to extend J_w if and only if it satisfies two conditions: 1) *index ordering constraint*: $q > r_d$. This condition enforces a unique traversal order by ensuring that items are appended into the itemset in increasing index order, thereby preventing duplicate itemset generation; 2) *attribute non-conflict constraint*: The attribute of $\mathcal{I}^c[q]$ must not match the attribute of any item already in J_w (Note that an item, e.g., $\forall \mathcal{I}^c[r] \in \mathcal{I}^c$, in our paper is denoted as an attribute-value pair as introduced in Sec. 3). This constraint naturally filters out invalid itemsets because a user's record contains exactly one value per attribute, meaning distinct items sharing the same attribute but having different values can never co-occur (i.e., yielding a true support count of 0). For each valid item $\mathcal{I}^c[q]$, the server creates a child node w_c whose itemset

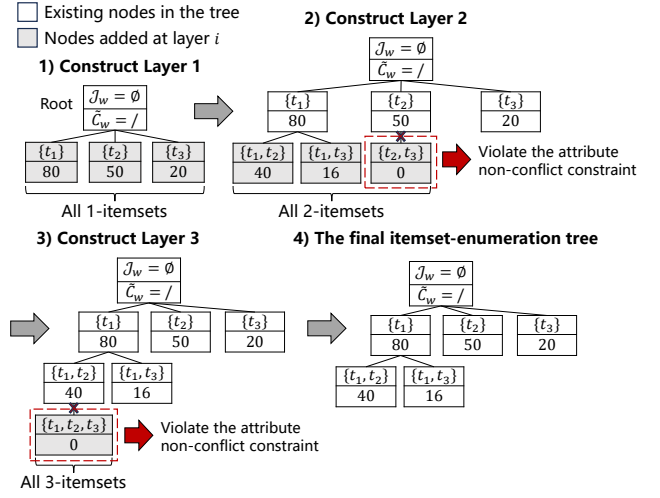


Figure 2: An example of constructing the itemset-enumeration tree from $\mathcal{I}^c$ of size 3, where we denote items $\mathcal{I}^c[r] \forall r \in [3]$ as t_r . From the root, the tree is constructed layer by layer, and each node records an itemset and its estimated support count. We assume items t_2 and t_3 contain the same attribute. Thus, any itemset containing both items violates the attribute non-conflict constraint. Therefore, $\{t_2, t_3\}$ and $\{t_1, t_2, t_3\}$ are not added to the tree.

is set as $J_{w_c} = J_w \cup \{\mathcal{I}^c[q]\}$. The server then needs to estimate the support count $\tilde{C}_{w_c}$ for this child node. Inspired by [60], we assume items are independent of each other and estimate $\tilde{C}_{w_c}$ as follows:

$$\tilde{C}_{w_c} = \hat{N} \cdot \prod_{e \in J_{w_c}} \frac{\tilde{c}(e)}{\hat{N}}, \quad (2)$$

where $\tilde{c}(e)$ represents the noisy support count of the item $e \in J_{w_c}$. $\hat{N} = \frac{1}{K} \sum_{i=1}^K \hat{N}_i$ is the average of perturbed numbers of users. Through this process, the server generates all valid itemsets of size $d+1$ formed by items in $\mathcal{I}^c$ at layer $d+1$. By expanding all the nodes in the tree, the server enumerates all the itemsets formed by the items in $\mathcal{I}^c$ and calculates their estimated support counts. An example of constructing $\mathcal{T}$ is illustrated in Figure 2.

Pruning Strategy. Although the aforementioned tree construction enumerates all itemsets formed by the items in $\mathcal{I}^c$, the search space is exceptionally large with $O(2^{K \cdot k})$. To mitigate the computational overhead and accelerate the search process, we introduce a dynamic pruning strategy during the tree construction. Specifically, when a node w generates a child node w_c , the server evaluates its estimated support count $\tilde{C}_{w_c}$. If $\tilde{C}_{w_c}$ is smaller than a threshold σ_{2k} , which means its corresponding itemset J_{w_c} is unlikely to be a highly frequent itemset, the server prunes the branch at w_c and halts any further expansion from this node. Note that this pruning is completely safe and does not discard any itemsets with potentially larger counts. According to Eq. (2), extending an itemset with a new item e simply multiplies the current estimated count by a fraction $\frac{\tilde{c}(e)}{\hat{N}} \leq 1$. Therefore, any child node generated from w_c is guaranteed to have an estimated count less than or equal to $\tilde{C}_{w_c}$. If $\tilde{C}_{w_c}$ is already smaller than σ_{2k} , all its child nodes will definitively fall below the threshold, making further search unnecessary.

Algorithm 2: Tree Construction and Coarse Pruning

Input: Reports from parties $\mathcal{P}_1, \dots, \mathcal{P}_K, \hat{N}_1, \dots, \hat{N}_K$, candidate size $2k$
Output: Top- $2k$ candidate itemsets with their estimated support counts C_{2k}

```

1 Aggregate  $\mathcal{P}_i$  into the unified sequence  $\mathcal{I}^c$  of size  $K \cdot k$ ;
2 Initialize a min-heap buffer  $H \leftarrow \emptyset$ , threshold  $\sigma_{2k} \leftarrow 0$ , and queue  $Q \leftarrow \emptyset$ ;
3 Initialize the root node  $w_{\text{root}}$  with  $J_{w_{\text{root}}} = \emptyset$ ;
4 for  $r \leftarrow 1$  to  $K \cdot k$  do
5   Create node  $w_r$  with itemset  $J_{w_r} = \{\mathcal{I}^c[r]\}$  and  $\tilde{C}_{w_r} = \tilde{c}(\mathcal{I}^c[r])$ ;
6   if  $|H| < 2k$  then
7     Insert  $(J_{w_r}, \tilde{C}_{w_r})$  into  $H$ ;
8   else
9     if  $|H| == 2k$  then  $\sigma_{2k} \leftarrow \min_{(J, \tilde{C}) \in H} \tilde{C}$ ;
10    if  $\tilde{C}_{w_r} > \sigma_{2k}$  then
11      Remove the itemset with the lowest count from  $H$  and insert
12       $(J_{w_r}, \tilde{C}_{w_r})$ ;
13     $\sigma_{2k} \leftarrow \min_{(J, \tilde{C}) \in H} \tilde{C}$ ;
14  end
15  Push node  $w_r$  into queue  $Q$ ;
16 end
17 while  $Q$  is not empty do
18   Pop node  $w$  from queue  $Q$ ;
19   for each item  $\mathcal{I}^c[q] \in \mathcal{I}^c$  do
20     if  $\mathcal{I}^c[q]$  is a valid item to extend  $J_w$  then
21       Create child node  $w_c$  with  $J_{w_c} = J_w \cup \{\mathcal{I}^c[q]\}$ ;
22       Calculate  $\tilde{C}_{w_c}$  via Eq. (2);
23       if  $\tilde{C}_{w_c} \geq \sigma_{2k}$  then
24         Remove the itemset with the lowest count from  $H$  and
25         insert  $(J_{w_c}, \tilde{C}_{w_c})$ ;
26          $\sigma_{2k} \leftarrow \min_{(J, \tilde{C}) \in H} \tilde{C}$ ;
27         Push node  $w_c$  into queue  $Q$ ;
28       end
29     end
30   end
31 end
32 return  $C_{2k} = \{(J, \tilde{C}) \mid (J, \tilde{C}) \in H\}$ ;

```

It remains to determine σ_{2k} . If σ_{2k} is too large, the server risks over-pruning and missing truly frequent itemsets; conversely, if σ_{2k} is set too small, the pruning condition becomes difficult to trigger, failing to effectively reduce the search space. To balance this, we propose adjusting σ_{2k} dynamically. From the beginning of the tree construction, the server maintains a buffer of size $2k$ (e.g., a min-heap) to store the top- $2k$ most frequent itemsets and their corresponding estimated support counts encountered so far. The threshold σ_{2k} is defined as the minimum count currently in this buffer. Whenever a newly expanded node yields a count larger than σ_{2k} , it is inserted into the buffer, the itemset with the lowest count is removed, and σ_{2k} is updated immediately. As the tree is constructed, σ_{2k} gradually rises, making the pruning condition increasingly strict while also preserving highly frequent itemsets. Upon completing the entire construction, the buffer inherently retains the top- $2k$ candidate itemsets along with their estimated counts.

Detailed Algorithm. Algorithm 2 summarizes the details of the server-side tree construction and coarse pruning. Initially, the central server aggregates the received items into the sequence $\mathcal{I}^c$ and initializes a min-heap buffer H , a pruning threshold σ_{2k} , and a queue Q to facilitate the layer-by-layer traversal (lines 1-2). To construct layer 1, the server creates child nodes w_r of the root with itemset J_{w_r} and estimated support counts $\tilde{C}_{w_r}$ (line 5). The server then updates the buffer H . If H contains fewer than $2k$ elements, the node is inserted directly, and σ_{2k} is initialized once the buffer is full; otherwise, the server compares the count against σ_{2k} to determine whether to replace the minimum element in H and update σ_{2k} accordingly (lines 6-14). These layer 1 nodes are subsequently pushed into the queue Q for further expansion

(line 15). During the layer-by-layer tree expansion, the server pops a node w from the queue and iterates over valid items $\mathcal{I}^c[q]$ to extend J_w (i.e., items satisfying the index ordering and attribute non-conflict constraints) (lines 18-29). For each valid extension, the server creates a child node w_c and derives its itemset J_{w_c} and estimated support count $\tilde{C}_{w_c}$ (lines 21-22). Following the pruning strategy, the server compares the count against the threshold. If $\tilde{C}_{w_c} \geq \sigma_{2k}$ which means the node survives the pruning, the node will be inserted into the buffer H , and the lowest-count itemset will be removed from H (line 24). Accordingly, the threshold σ_{2k} will be updated (line 25). And the node will be pushed into the queue for further exploration (line 26). Otherwise, the branch is pruned. Finally, the algorithm terminates when the entire search space has been traversed or pruned, returning C_{2k} (i.e., top- $2k$ candidate itemsets and their estimated support counts) and sending it to all the parties for the second interaction (line 31).

4.3 Refined Selection of Top- k Itemsets

4.3.1 Party-Side Precise Sketching. Upon receiving C_{2k} , the data parties initiate the second interaction to accurately determine the final top- k frequent itemsets with DP-FM sketches. The party-side precise sketching consists of two stages as follows.

Local Itemset Selection. For the convenience of derivation, we denote the received candidates as $C_{2k} = \{(J^{(r)}, \tilde{C}^{(r)}) \mid r \in [2k]\}$, where $(J^{(r)}, \tilde{C}^{(r)})$ is the r -th itemset and its estimated support count sorted in descending order of the support count. Each data party $i \in [K]$ first needs to choose local itemsets (i.e., sets formed by local items $t \in I_i$) to be encoded into DP-FM sketches. Denote the set formed by these local itemsets in party i as $\Omega_i \subseteq 2^{I_i}$, where 2^{I_i} is the power set of I_i . Note that, unlike only extracting *local items* in the first interaction, here we propose to extract *local itemsets* from I_i for party i to preserve intra-party attribute correlations (i.e., the co-occurrence of multiple local items within party i) and ensure accurate cross-party intersection estimation at the server.

Specifically, party i first traverses through C_{2k} and extracts local portion $J_i^{(r)}$ from each candidate itemset $J^{(r)}$ by taking the intersection, i.e., $J_i^{(r)} = J^{(r)} \cap I_i$. To prioritize the most promising candidates, each local portion $J_i^{(r)}$ is then associated with the support count $\tilde{C}^{(r)}$. Because our ultimate goal is to output the top- k itemsets, the server does not require sketches for all possible local portions. Thus, party i only selects k unique local portions with the highest associated support counts to form the set $\Omega_i = \{J_i^1, \dots, J_i^k\}$. Note that we utilize the estimated support counts $\tilde{C}^{(r)}$ provided by the server to select Ω_i , rather than re-evaluating counts locally. This is because re-evaluating counts locally would consume additional privacy budget. And relying on $\tilde{C}^{(r)}$ ensures a globally optimal selection, since itemsets that are highly frequent locally do not necessarily align with the globally top- k itemsets across all parties.

DP-FM Sketch Generation. After determining Ω_i , each data party $i \in [K]$ constructs DP-FM sketches for all local itemsets within Ω_i . Specifically, for each local itemset $J_i^p \in \Omega_i$ (where $p \in [k]$), party i first collects the identifiers of users whose local records contain the itemset J_i^p , forming a local user set $\mathcal{X}_i^p = \{\text{id}_l \mid J_i^p \subseteq S_l^i, l \in [N]\}$. Then, party i processes $\mathcal{X}_i^p$ via Algorithm 1 repeated h times to generate a DP-FM sketch vector

$\mathcal{M}_i^p = \{\alpha_i^{p,1}, \dots, \alpha_i^{p,h}\}$. Note that here all K parties share the same h hash keys $\zeta_1, \dots, \zeta_h$ to generate the DP-FM sketches for each set $\mathcal{X}_i^p$. To satisfy DP, party i evenly divides its privacy budget ϵ_2 for the second interaction among the k itemsets. By setting the privacy budget in Algorithm 1 to $\epsilon' = \frac{\epsilon_2/k}{4\sqrt{h \log(k/\delta)}}$, Lemma 2.2 ensures each

$\mathcal{M}_i^p$ satisfies $(\epsilon_2/k, \delta/k)$ -DP. Thus, reporting $\mathcal{M}_i^p$ for k itemsets satisfies (ϵ_2, δ) -DP for party i . Finally, party i sends the selected local itemsets and their corresponding DP-FM sketch vectors, denoted as $\mathcal{Q}_i = \{(J_i^p, \mathcal{M}_i^p) \mid p \in [k]\}$, to the central server.

4.3.2 Server-Side Refined Estimation and Mining. Upon receiving the reports $\mathcal{Q}_1, \dots, \mathcal{Q}_K$ from all data parties, the central server needs to accurately estimate the support count for each candidate itemset $J^{(r)}$ in $\mathcal{C}_{2k}$, and ultimately determine the final top- k frequent itemsets. This refined estimation and mining process proceeds in three stages. 1) The server performs a *valid candidate matching* to identify which candidates $J^{(r)}$ can have their support counts refined from the received reports. 2) The server conducts *sketch support estimation* to estimate the support of each valid candidate. Here we employ a Maximum A Posteriori (MAP) method to reduce the estimation error. 3) The server executes *top- k itemset selection* to output the refined globally frequent itemsets. Note that in the second interaction, the server does not need to adopt the itemset-enumeration tree since it only needs to select the top- k frequent itemsets from a small size of $2k$ candidate itemsets.

1) Valid Candidate Matching. After receiving the reports, the server first needs to determine whether the support count of a candidate itemset $J^{(r)}$ can be calculated. Since each party i only uploads sketches for a limited set of k local itemsets, a candidate itemset $J^{(r)}$ might lack the required local portion J_i^p from a certain party, making its support count calculation impossible. Specifically, for each candidate itemset $J^{(r)}$, the server first identifies a valid matching set of local itemsets, denoted as $\mathcal{B}^{(r)} \subseteq [K] \times [k]$. Each element (i, p) in $\mathcal{B}^{(r)}$ indicates that the p -th local itemset reported by party i (i.e., J_i^p) is utilized to assemble the candidate itemset $J^{(r)}$. A candidate is considered valid if and only if its matched local itemsets $J_i^p \forall (i, p) \in \mathcal{B}^{(r)}$ can reconstruct the candidate itemset $J^{(r)}$, i.e., $J^{(r)} = \bigcup_{(i,p) \in \mathcal{B}^{(r)}} J_i^p$. If no such valid matching set $\mathcal{B}^{(r)}$ can be found for a candidate $J^{(r)}$, it implies that the required local itemsets were not reported by the corresponding data party. Consequently, the server will discard $J^{(r)}$. Note that because all K parties select k local portions corresponding to the highest estimated counts from $\mathcal{C}_{2k}$, at least the top- k candidate itemsets from the coarse screening (i.e., the first interaction) are guaranteed to be valid. Thus, this discarding will never result in fewer than k remaining candidates. Without loss of generality, in the subsequent stages, we assume all the candidate itemsets $J^{(r)} \forall r \in [2k]$ in $\mathcal{C}_{2k}$ possess a valid matching set $\mathcal{B}^{(r)}$ and can be successfully estimated.

2) Sketch Support Estimation. For each valid candidate itemset $J^{(r)}$, the server aims to estimate its refined support count $\hat{C}^{(r)}$. If $|\mathcal{B}^{(r)}| = 1$ (i.e., $\mathcal{B}^{(r)} = \{(i, p)\}$), $\hat{C}^{(r)}$ is the estimation of the local cardinality $n_i^p = |\mathcal{X}_i^p|$ using $\mathcal{M}_i^p$. Unlike the standard approach in Sec. 2.2, here we propose *MAP-based local cardinality estimation* to estimate n_i^p and reduce the estimation error. More importantly, if $|\mathcal{B}^{(r)}| > 1$, a user's global record S^i contains $J^{(r)}$ if and only if the user identifier id_i exists in all corresponding local user sets

$\mathcal{X}_i^p \forall (i, p) \in \mathcal{B}^{(r)}$. Thus, calculating $\hat{C}^{(r)}$ is equivalent to estimating the intersection cardinality of $\mathcal{X}_i^p$ for all $(i, p) \in \mathcal{B}^{(r)}$, i.e., estimating $|\bigcap_{(i,p) \in \mathcal{B}^{(r)}} \mathcal{X}_i^p|$ as $\hat{C}^{(r)}$. To enable precise estimation of this intersection cardinality, we adopt two steps: *MAP-based local cardinality estimation*, and *MAP-based intersection cardinality estimation*, discarding the intersection estimation method introduced in Sec. 2.2.

• MAP-based Local Cardinality Estimation. In this step, the server aims to estimate the local cardinality $n_i^p = |\mathcal{X}_i^p|$ as $\hat{n}_i^p$ for a specific pair $(i, p) \in \mathcal{B}^{(r)}$, based solely on the observed DP-FM sketch vector $\mathcal{M}_i^p = \{\alpha_i^{p,1}, \dots, \alpha_i^{p,h}\}$. Note that if $|\mathcal{B}^{(r)}| = 1$, this is the only step for $J^{(r)}$ to estimate its $\hat{C}^{(r)}$. After this step, $\hat{C}^{(r)}$ is directly set to $\hat{n}_i^p$. To estimate the local cardinality n_i^p , we first establish the likelihood of observing a specific hash value. Recall from Algorithm 1 that the DP-FM sketch introduces $n_p = \lceil 1/(e^{\epsilon'} - 1) \rceil$ phantom elements and enforces a lower bound $\alpha_{\min} = \lceil \log_{1+\gamma}(1/(1 - e^{-\epsilon'})) \rceil$. For the ideal geometric-value hash functions $H_{\zeta_1}, \dots, H_{\zeta_h}$ whose hashed values follow i.i.d. Geometric($\frac{\gamma}{1+\gamma}$), the cumulative distribution function (CDF) of a single hashed value can be calculated as $F(x) = 1 - (1 + \gamma)^{-x}$ for $x \geq 1$, and 0 otherwise. Based on these properties, the likelihood of a single DP-FM output $\alpha_i^{p,g} \in \mathcal{M}_i^p$ can be calculated as follows:

THEOREM 4.1 (LIKELIHOOD OF DP-FM OUTPUT). Assume the cardinality of the local user set $\mathcal{X}_i^p$ is $n_i^p = n$. Let $\alpha_i^{p,g}$ be the g -th observed hash value in $\mathcal{M}_i^p$. The conditional probability mass function (PMF) of $\alpha_i^{p,g}$, i.e., $P_i^p(a \mid n) = \Pr(\alpha_i^{p,g} = a \mid n_i^p = n)$, can be calculated as $P_i^p(a \mid n) = F_i^p(a \mid n) - F_i^p(a - 1 \mid n)$, where $F_i^p(x \mid n) = \Pr(\alpha_i^{p,g} \leq x \mid n_i^p = n)$ is the conditional CDF of the DP-FM output, calculated as follows:

$$F_i^p(x \mid n) = \begin{cases} 0, & x < \alpha_{\min}, \\ F(x)^{n+n_p}, & x \geq \alpha_{\min}. \end{cases} \quad (3)$$

PROOF. See Appendix A.1 in our technical report [3]. $\square$

Given that the h hash values in the sketch vector $\mathcal{M}_i^p$ are generated independently using different hash keys, the joint likelihood $\Pr(\mathcal{M}_i^p \mid n)$ of observing the entire sketch vector $\mathcal{M}_i^p$ is the product of their individual likelihoods, i.e., it can be calculated as $\Pr(\mathcal{M}_i^p \mid n) = \prod_{g=1}^h P_i^p(\alpha_i^{p,g} \mid n)$. To perform Bayesian inference, we must also specify the prior distribution $\Pr(n_i^p = n)$ of the local cardinality n_i^p . Since the server has no prior knowledge regarding the local user-set cardinality other than the fact that it is bounded by the total number of users N , we adopt a uniform prior distribution as follows. Note that, to satisfy DP, the server cannot get the precise number of users N . However, it can be estimated as $\hat{N} = \frac{1}{K} \sum_{i=1}^K \hat{N}_i$ with the first-round reports of data parties.

Definition 4.2 (Prior Distribution of Local Cardinality). The prior probability of the local cardinality n_i^p is uniformly distributed over the range $\{0, 1, \dots, \hat{N}\}$, i.e., $\Pr(n_i^p = n) = \frac{1}{\hat{N}+1}, \forall n \in \{0, 1, \dots, \hat{N}\}$.

According to Bayes' theorem, the posterior probability $\Pr(n_i^p = n \mid \mathcal{M}_i^p)$ is proportional to the product of the prior and the joint likelihood, i.e., $\Pr(n_i^p = n \mid \mathcal{M}_i^p) \propto \Pr(n_i^p = n) \cdot \Pr(\mathcal{M}_i^p \mid n) = \frac{1}{\hat{N}+1} \prod_{g=1}^h P_i^p(\alpha_i^{p,g} \mid n)$. The server then applies MAP estimation to

find the most probable cardinality and estimate n_i^p as $\hat{n}_i^p$:

$$\hat{n}_i^p = \arg \max_{n \in \{0, \dots, \hat{N}\}} \Pr(n_i^p = n \mid \mathcal{M}_i^p)$$

$$= \arg \max_{n \in \{0, \dots, \hat{N}\}} \left\{ \log \left(\frac{1}{\hat{N} + 1} \right) + \sum_{g=1}^h \log P_i^p(\alpha_i^{p,g} \mid n) \right\}. \quad (4)$$

Here we calculate the MAP estimate $\hat{n}_i^p$ in the logarithmic domain to prevent numerical underflow caused by multiplying multiple small probability values. If $|\mathcal{B}^{(r)}| = 1$, the refined support count of the candidate is directly determined as $\hat{C}^{(r)} = \hat{n}_i^p$. If $|\mathcal{B}^{(r)}| > 1$, the server will estimate the local cardinalities $\hat{n}_i^p$ for all the pairs $(i, p) \in \mathcal{B}^{(r)}$. These cardinalities $\hat{n}_i^p \forall (i, p) \in \mathcal{B}^{(r)}$ will serve as the basis for the subsequent intersection cardinality estimation.

• **MAP-based Intersection Cardinality Estimation.** For a candidate itemset $J^{(r)}$ whose $|\mathcal{B}^{(r)}|$ is larger than 1, its true support count $C^{(r)}$ is exactly the intersection cardinality of all corresponding local user sets, i.e., $C^{(r)} = |\bigcap_{(i,p) \in \mathcal{B}^{(r)}} \mathcal{X}_i^p|$. Unlike n_i^p , $C^{(r)}$ usually cannot achieve all the values in $\{0, 1, \dots, N\}$. Thus, to estimate this cardinality, we first establish its feasible range as follows.

THEOREM 4.3 (BOUNDS OF INTERSECTION CARDINALITY). *The true intersection cardinality $C^{(r)}$ is bounded by $\max\{0, \sum_{(i,p) \in \mathcal{B}^{(r)}} n_i^p - (|\mathcal{B}^{(r)}| - 1)N\} \leq C^{(r)} \leq \min_{(i,p) \in \mathcal{B}^{(r)}} n_i^p$.*

PROOF. See Appendix A.2 in our technical report [3]. $\square$

The total number of users N and the local cardinalities n_i^p in the bounds are unknown to the server. However, with the previous step, the server can estimate $n_i^p \forall (i, p) \in \mathcal{B}^{(r)}$ as $\hat{n}_i^p$ and N can be estimated as $\hat{N} = \frac{1}{K} \sum_{i=1}^K \hat{N}_i$. Thus, we substitute N and n_i^p with $\hat{N}$ and $\hat{n}_i^p$ to approximate the upper bound as $\hat{U}^{(r)} = \min_{(i,p) \in \mathcal{B}^{(r)}} \hat{n}_i^p$ and the lower bound as $\hat{L}^{(r)} = \max\{0, \sum_{(i,p) \in \mathcal{B}^{(r)}} \hat{n}_i^p - (|\mathcal{B}^{(r)}| - 1)\hat{N}\}$. The server then can establish the feasible range of $C^{(r)}$ as $\mathcal{N}^{(r)} = \{\hat{L}^{(r)}, \hat{L}^{(r)} + 1, \dots, \hat{U}^{(r)}\}$. Based on this, we specify a uniform prior distribution $\Pr(C^{(r)} = n)$ over this range for $C^{(r)}$.

Definition 4.4 (Prior Distribution of Intersection Cardinality). The prior probability of the intersection cardinality $C^{(r)}$ is uniformly distributed over $\mathcal{N}^{(r)}$, i.e., $\Pr(C^{(r)} = n) = \frac{1}{|\mathcal{N}^{(r)}|}, \forall n \in \mathcal{N}^{(r)}$.

Next, we establish the likelihood of observing the multi-party DP-FM sketches given an intersection size $C^{(r)} = n \in \mathcal{N}^{(r)}$. Since the h hash values in each sketch vector are generated independently, we first focus on the g -th observed hash values across all relevant parties (i.e., the g -th values in the DP-FM sketches $\mathcal{M}_i^p$ for all $(i, p) \in \mathcal{B}^{(r)}$), denoted as $\{\alpha_i^{p,g}\}_{(i,p) \in \mathcal{B}^{(r)}}$. The exact joint likelihood for the g -th values is calculated as $\Pr(\{\alpha_i^{p,g}\}_{(i,p) \in \mathcal{B}^{(r)}} \mid C^{(r)} = n, \{n_i^p = \hat{n}_i^p\}_{(i,p) \in \mathcal{B}^{(r)}})$. Note that we condition on $\{n_i^p = \hat{n}_i^p\}$ because the distribution of a DP-FM output inherently depends on the cardinality of its corresponding local user set $\mathcal{X}_i^p$, which we approximate using the MAP estimates obtained in the previous step. However, this exact joint likelihood is computationally intractable. This difficulty arises because users who belong to multiple (but not all) local sets $\mathcal{X}_i^p$ will generate identical hash values across corresponding sketches. To calculate the exact joint probability, the server needs to know the exact number of users in every such partially overlapping region among the local sets $\mathcal{X}_i^p$, which is completely unknown. To

Algorithm 3: Server-Side Refined Estimation and Mining

Input: Candidate itemsets $\mathcal{C}_{2k}$, reports $Q_1, \dots, Q_K$, target size k
Output: Top- k frequent itemsets $\mathcal{F}_k$

```

1  $\mathcal{R} \leftarrow \emptyset$ ;
2 for each  $(J^{(r)}, \hat{C}^{(r)}) \in \mathcal{C}_{2k}$  do
3   Find matching set  $\mathcal{B}^{(r)}$  using  $Q_1, \dots, Q_K$ ;
4   if  $J^{(r)} = \bigcup_{(i,p) \in \mathcal{B}^{(r)}} J_i^p$  then
5     if  $|\mathcal{B}^{(r)}| == 1$  then
6       Calculate  $\hat{C}^{(r)}$  via Eq. (4);
7     else
8       Calculate  $\hat{n}_i^p$  for all  $(i, p) \in \mathcal{B}^{(r)}$  via Eq. (4);
9       Calculate  $\hat{C}^{(r)}$  via Eq. (6);
10    end
11     $\mathcal{R} \leftarrow \mathcal{R} \cup \{(J^{(r)}, \hat{C}^{(r)})\}$ ;
12  end
13 end
14 Sort  $\mathcal{R}$  in descending order of  $\hat{C}^{(r)}$  and select top- $k$  itemsets to form  $\mathcal{F}_k$ ;
15 return  $\mathcal{F}_k$ ;

```

enable the estimation, we apply an approximation condition, denoted as $\mathcal{E}_{\text{disjoint}}$, which assumes that after removing the common intersection $\mathcal{X}_{\cap}^{(r)} = \bigcap_{(i,p) \in \mathcal{B}^{(r)}} \mathcal{X}_i^p$, the remaining parts of the different local user sets are mutually disjoint. Formally, this condition assumes that $(\mathcal{X}_i^p \setminus \mathcal{X}_{\cap}^{(r)}) \cap (\mathcal{X}_{i'}^{p'} \setminus \mathcal{X}_{\cap}^{(r)}) = \emptyset, \forall (i, p) \neq (i', p') \in \mathcal{B}^{(r)}$. Under this condition, the approximate conditional likelihood for the g -th values is $\mathcal{L}_g^{(r)}(n) = \Pr(\{\alpha_i^{p,g}\}_{(i,p) \in \mathcal{B}^{(r)}} \mid C^{(r)} = n, \{n_i^p = \hat{n}_i^p\}_{(i,p) \in \mathcal{B}^{(r)}}, \mathcal{E}_{\text{disjoint}})$, which can be calculated as follows.

THEOREM 4.5 (APPROXIMATE LIKELIHOOD OF MULTI-PARTY OUTPUTS). *Given an intersection size $n \in \mathcal{N}^{(r)}$, let $t_{i,p}^{g,W} = \alpha_i^{p,g} - \mathbb{I}((i, p) \in W)$ for any subset $W \subseteq \mathcal{B}^{(r)}$, where $\mathbb{I}(\cdot)$ is the indicator function. Let $t_{\min}^{g,W} = \min_{(i,p) \in \mathcal{B}^{(r)}} t_{i,p}^{g,W}$. The approximate conditional likelihood $\mathcal{L}_g^{(r)}(n)$ is calculated as $\mathcal{L}_g^{(r)}(n) = \sum_{W \subseteq \mathcal{B}^{(r)}} (-1)^{|W|} L(W, n)$, where $L(W, n)$ is calculated as follows:*

$$L(W, n) = \begin{cases} 0, & t_{\min}^{g,W} < \alpha_{\min}, \\ F(t_{\min}^{g,W})^n \prod_{(i,p) \in \mathcal{B}^{(r)}} F(t_{i,p}^{g,W})^{\hat{n}_i^p - n + n_p}, & t_{\min}^{g,W} \geq \alpha_{\min}, \end{cases} \quad (5)$$

where $F(x)$ is the CDF of a single hashed value and n_p is the number of phantom elements.

PROOF. See Appendix A.3 in our technical report [3]. $\square$

Since the hash functions are independent across the h repetitions, the overall joint likelihood of observing all sketches can be calculated as $\prod_{g=1}^h \mathcal{L}_g^{(r)}(n)$. According to Bayes' theorem, the posterior probability $\Pr(C^{(r)} = n \mid \{\mathcal{M}_i^p\}_{(i,p) \in \mathcal{B}^{(r)}})$ is approximately proportional to the product of the prior and the joint likelihood $\Pr(C^{(r)} = n) \prod_{g=1}^h \mathcal{L}_g^{(r)}(n) = \frac{1}{|\mathcal{N}^{(r)}|} \prod_{g=1}^h \mathcal{L}_g^{(r)}(n)$. Finally, the server applies MAP estimation to find the most probable intersection cardinality and estimate $C^{(r)}$ as $\hat{C}^{(r)}$. Similar to Eq. (4), we calculate $\hat{C}^{(r)}$ in the logarithmic domain to prevent underflow:

$$\hat{C}^{(r)} = \arg \max_{n \in \mathcal{N}^{(r)}} \left\{ \log \left(\frac{1}{|\mathcal{N}^{(r)}|} \right) + \sum_{g=1}^h \log \mathcal{L}_g^{(r)}(n) \right\}. \quad (6)$$

3) Top- k Itemset Selection. After refining the support counts $\hat{C}^{(r)}$ for all valid candidates $J^{(r)}$, the central server sorts them in descending order and outputs the top- k itemsets with the highest counts. Note that unlike the first interaction, we do not construct

an enumeration tree here because directly evaluating $2k$ candidate itemsets is already efficient. Algorithm 3 summarizes the entire server-side refined estimation and mining procedure. Specifically, the server first identifies the matching set $\mathcal{B}^{(r)}$ for each candidate (Line 3). For valid candidates, it estimates $\hat{C}^{(r)}$ using Eq. (4) if $|\mathcal{B}^{(r)}| = 1$, or Eq. (6) if $|\mathcal{B}^{(r)}| > 1$ (Lines 4-12). Finally, it sorts all refined candidates and outputs the top- k itemsets (Lines 14-15).

5 THEORETICAL ANALYSIS

Privacy Analysis. PriVFim satisfies DP as follows.

THEOREM 5.1 (PRIVACY GUARANTEE OF PRIVFIM). *Let ϵ_1 and ϵ_2 be the privacy budgets used in the first and second interactions, respectively, and $\epsilon = \epsilon_1 + \epsilon_2$. Let $0 < \delta < 1$, and assume that the privacy budget ϵ_2 satisfies $\epsilon_2/k \leq 2 \log(k/\delta)$ according to Lemma 2.2. Then PriVFim satisfies (ϵ, δ) -DP.*

PROOF. See Appendix A.4 in our technical report [3]. $\square$

Utility Analysis. PriVFim uses MAP estimation for both local cardinalities and intersection cardinalities. For local cardinality estimation of a local set $\mathcal{X}_i^p$, the likelihood in repetition g under a possible cardinality $n \in \{0, \dots, \hat{N}\}$ is $P_i^p(\alpha_i^{p,g} | n)$, as given by Theorem 4.1. For intersection cardinality estimation of a candidate $J^{(r)}$ involving multiple local sets, the likelihood under a possible intersection cardinality $n \in \mathcal{N}^{(r)}$ is $\mathcal{L}_g^{(r)}(n)$, as given by Theorem 4.5. To cover both cases, let $\mathcal{S}$ be the set of cardinalities considered by MAP, let c be the true cardinality, let c_{MAP} be its MAP estimate, and let $\ell_g(n)$ be the likelihood in repetition g . For local cardinality estimation, these quantities are $\mathcal{S} = \{0, \dots, \hat{N}\}$, $c = n_i^p$, $c_{\text{MAP}} = \hat{n}_i^p$, and $\ell_g(n) = P_i^p(\alpha_i^{p,g} | n)$. For intersection cardinality estimation, they are $\mathcal{S} = \mathcal{N}^{(r)}$, $c = C^{(r)}$, $c_{\text{MAP}} = \hat{C}^{(r)}$, and $\ell_g(n) = \mathcal{L}_g^{(r)}(n)$. The following theorem gives the error bound for MAP estimation.

THEOREM 5.2 (ERROR BOUND FOR MAP ESTIMATION). *Assume that $c \in \mathcal{S}$, the prior on $\mathcal{S}$ is uniform, and the h repetitions of DP-FM sketches are independent. For intersection estimation, further assume that $\hat{n}_i^p = n_i^p$ for all $(i, p) \in \mathcal{B}^{(r)}$ and that $\mathcal{E}_{\text{disjoint}}$ holds. For any $n \in \mathcal{S}$ satisfying $|n - c| \geq q$, define $Z_{n,g} = \log(\ell_g(c)/\ell_g(n))$. If $|Z_{n,g}| \leq B$ and $\mathbb{E}[Z_{n,g}] \geq \Delta > 0$ for every such n and g , then $\Pr(|c_{\text{MAP}} - c| \geq q) \leq |\mathcal{S}| \exp(-h\Delta^2/(2B^2))$.*

PROOF. See Appendix A.5 in our technical report [3]. $\square$

Here $Z_{n,g}$ compares the likelihoods of the true cardinality and an incorrect cardinality n . The theorem shows that the MAP error probability decreases exponentially with h when the true cardinality has a higher likelihood on average. The following theorem also gives a utility guarantee for the complete mining process. Let $\mathcal{J}_{2k} = \{J^{(r)} \mid r \in [2k]\}$ be the candidate itemsets and let $\mathcal{T}_k$ be the true global top- k itemsets. Define $\Gamma = \min_{X \in \mathcal{T}_k} C_X - \max_{Y \in \mathcal{J}_{2k} \setminus \mathcal{T}_k} C_Y$.

THEOREM 5.3 (TOP- k MINING OF PRIVFIM). *Assume that $\mathcal{T}_k \subseteq \mathcal{J}_{2k}$, $\Gamma > 0$, and $\mathcal{E}_{\text{disjoint}}$ holds for every candidate involving multiple local sets. Suppose the conditions of Theorem 5.2 hold with $q = 1$ for every local estimate and with $q = \Gamma/2$ for every intersection estimate, using common constants B and Δ . Then PriVFim returns $\mathcal{T}_k$ with probability at least $1 - (K + 2)k(\hat{N} + 1) \exp(-h\Delta^2/(2B^2))$.*

PROOF. See Appendix A.6 in our technical report [3]. $\square$

Computational Cost. Let M be the total number of attributes, let T be the number of nodes visited during coarse screening, let $s_r = |\mathcal{B}^{(r)}|$, and let $\mathcal{R}_\cap = \{r \in [2k] \mid s_r > 1\}$ contain the candidates that require intersection estimation. The overall computational cost is $O(NM + T \log k + KkhN + \sum_{r \in \mathcal{R}_\cap} h|\mathcal{N}^{(r)}|s_r 2^{s_r})$. In the worst case, $T \leq 2^{Kk}$ and $s_r \leq K$, while coarse screening reduces T in practice.

Communication Cost. Let ℓ_{max} be the maximum number of items contained in a candidate itemset. Assuming that sending a single item, noisy count, and DP-FM sketch value requires an $O(1)$ cost, sending a single candidate itemset incurs $O(\ell_{\text{max}})$ cost. Thus, the overall communication cost in the two rounds of interaction between data parties and the server amounts to $O(Kk(\ell_{\text{max}} + h))$.

6 EXPERIMENTAL EVALUATION

6.1 Experimental Setup

6.1.1 Datasets. We use five real-world datasets. Here we briefly introduce them, providing the details in Appendix B in [3].

- **Bank** [42] comprises records from telephone marketing campaigns of a Portuguese bank. There are $N = 41,176$ users with $M = 18$ attributes. Different attributes have different domain sizes s_i^j where $2 \leq s_i^j \leq 26$, resulting in $|I| = 144$ distinct items in total.

- **Census** [12] contains demographic data from the U.S. Census Bureau's surveys. The dataset has $N = 299,285$ users with $M = 40$ attributes. Different attributes have different domain sizes s_i^j where $2 \leq s_i^j \leq 52$, resulting in $|I| = 567$ distinct items in total.

- **DefaultCredit** [19] contains credit card client data from Taiwan, including demographic information, credit data, repayment history, and bill statements. The dataset has $N = 30,000$ users and $M = 23$ attributes. After discretizing the numerical attributes, s_i^j ranges from 2 to 11, giving $|I| = 229$ distinct items in total.

- **MiniBooNE** [41] consists of particle measurements from the neutrino experiment. This dataset has $N = 130,064$ users with $M = 50$ attributes. Different attributes have different domain sizes s_i^j with $5 \leq s_i^j \leq 10$, giving $|I| = 494$ distinct items in total.

- **PokerHand** [11] describes poker hands drawn from a standard deck of 52 cards. It has $N = 1,025,010$ users and $M = 10$ attributes, where five suit attributes have $s_i^j = 4$ values and five rank attributes have $s_i^j = 13$ values, resulting in $|I| = 85$ items in total.

6.1.2 Compared Methods. We compare PriVFim with the following five baselines for vertical federated top- k itemset mining.

- **IE-Other** [37] adopts the PriVFim framework but replaces the MAP-based intersection estimation method with the one introduced in Sec. 2.2, where the complement of the set to be intersected is obtained by subtracting the set from the universal user set $[N]$.

- **IE-Full** [37] is similar to IE-Other, also replacing the MAP-based method with the inclusion-exclusion-based method in Sec. 2.2. However, the complement of the set to be intersected is obtained by the union of FM sketches of different candidate itemsets.

- **FO** [58] replaces DP-FM sketches with Optimized Unary Encoding (OUE), a type of Frequency Oracle (FO) under local DP.

- **NIPP-FIM** [73] is a cryptographic method which uses Boolean circuits to calculate intersections and support counts.

- **VPP-OFIM** [72] is also a cryptographic method that uses two servers with transformed transactions, artificial records, and validity indicators encrypted with Paillier to select the top- k itemsets.

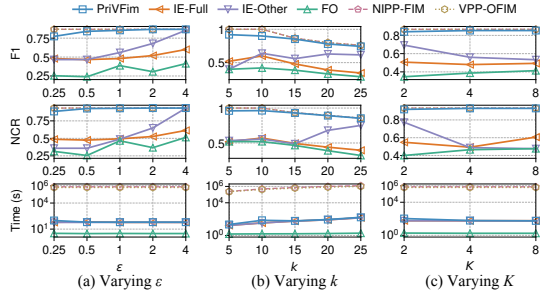


Figure 3: F1, NCR, and running time varying ϵ , k and K (Census dataset).

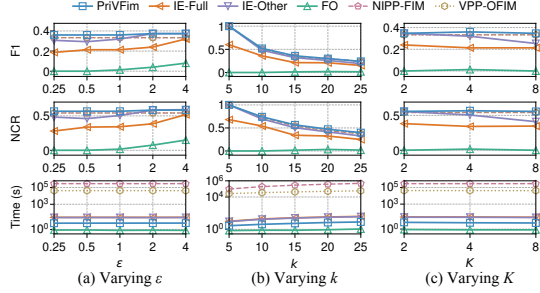


Figure 4: F1, NCR, and running time varying ϵ , k and K (MiniBooNE dataset).

6.1.3 Parameter Settings. We set the default privacy budget to $\epsilon = 1$ and $\delta = 10^{-5}$, and the number of DP-FM sketch repetitions to $h = 2048$. We set the default number of data parties to $K = 4$ to mine the top- $k = 15$ frequent itemsets. Each data party holds the same number of attributes, i.e., $m_i = \lfloor M/K \rfloor$. The default domain size s_i^j is set as in Sec. 6.1.1, with different datasets (and attributes) having different domain sizes. Detailed s_i^j for each dataset is provided in Appendix B in [3]. The default number of users N is set to the number of users in the corresponding dataset. All experiments are conducted on an Intel Xeon Platinum 8352V server at 2.10 GHz.

6.1.4 Evaluation Metrics. We evaluate mining accuracy using F1 score and Normalized Cumulative Rank (NCR), and computational efficiency using running time. The F1 score is defined as $F1 = \frac{2pr}{p+r}$, where $p = |\mathcal{F}_k \cap \mathcal{T}_k|/|\mathcal{F}_k|$ is the precision and $r = |\mathcal{F}_k \cap \mathcal{T}_k|/|\mathcal{T}_k|$ is the recall. $\mathcal{T}_k$ denotes the true top- k itemsets and $\mathcal{F}_k$ denotes the estimated top- k itemsets. The NCR assigns a higher penalty to missing the most frequent value than to others and is defined as $NCR = \frac{\sum_{X \in \mathcal{F}_k} q(X)}{\sum_{X \in \mathcal{T}_k} q(X)}$, where $q(X) = k - \text{rank}(X) + 1$ if X is ranked within the true top- k itemsets, and $q(X) = 0$ otherwise. $\text{rank}(X)$ denotes the position of X in the true ranking. A larger F1 or NCR value indicates better mining accuracy. Running time measures how many seconds it takes to execute a mining algorithm.

6.2 Experimental Results

6.2.1 Main Results. Figures 3 and 4 show the F1, NCR, and running time of different methods on Census and MiniBooNE, respectively. In terms of F1 and NCR, PrivFim achieves the best performance among the DP methods in most settings, followed by IE-Other and IE-Full. This is because PrivFim exploits the MAP-based method to estimate support counts. IE-Other is better than

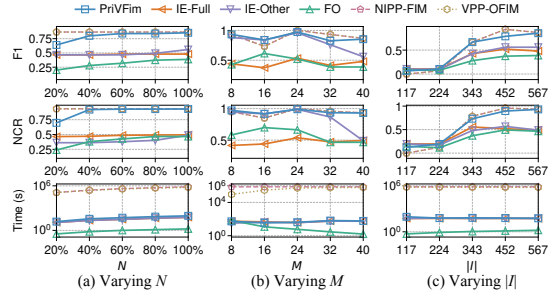


Figure 5: F1, NCR, and running time varying N , M , and $|I|$ (Census dataset).

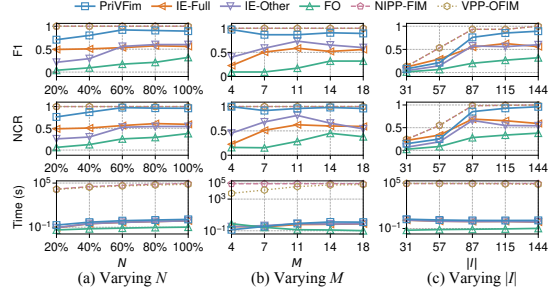


Figure 6: F1, NCR, and running time varying N , M , and $|I|$ (Bank dataset).

IE-Full because it directly obtains the complementary set of the set to be intersected, which leaves more budget for each sketch. FO achieves lower F1 and NCR because it requires each user to perturb their data locally, thus introducing more noise. As for running time, FO is the fastest. PrivFim has a running time similar to that of IE-Other and IE-Full. While achieving similar accuracy to PrivFim, the cryptographic methods are much slower than PrivFim.

6.2.2 Impact of ϵ . Figures 3(a) and 4(a) show the F1, NCR, and running time of different methods as the privacy budget ϵ varies. Note that the performance of the cryptographic methods does not change with ϵ . PrivFim achieves the highest F1 and NCR in most settings. As ϵ increases, its F1 and NCR improve since a larger ϵ means less noise is introduced into the local top- k item counts in the first round of interaction and DP-FM sketches in the second round of interaction. IE-Other and IE-Full also improve with ϵ and approach PrivFim as a larger ϵ makes their complement estimates more accurate. Running time remains stable when varying ϵ .

6.2.3 Impact of k . Figures 3(b) and 4(b) show the F1, NCR, and running time of different methods as k varies. As k increases, the F1 and NCR of PrivFim decrease because estimation errors accumulate as more itemsets are included into the final output. This decline is more pronounced on MiniBooNE, which has more attributes and thus a larger space of itemsets, making coarse screening more likely to miss true top- k itemsets as k increases. IE-Other and IE-Full are less sensitive to k , while their performance is much worse than that of PrivFim. Besides, running time increases with k increasing.

6.2.4 Impact of K . Figures 3(c) and 4(c) show the impact of varying the number of data parties K , while keeping the total number of attributes fixed. The mining accuracy of IE-Other and IE-Full becomes worse as the number of data parties increases. This is

because more data parties mean more DP-FM sketches to intersect, thus introducing more noise. In contrast, PriVFim maintains nearly unchanged F1 and NCR across different K , showing that it is less sensitive to how attributes are distributed among data parties. Running time of all the methods changes little while varying K .

6.2.5 Impact of N . To evaluate the impact of N , we sample different subsets of users, from 20% to 100%, in the datasets. Figures 5(a) and 6(a) show the results. PriVFim achieves the highest F1 and NCR among DP-based methods, and both improve as N increases. This is because, in the first interaction, the true support counts grow with N while the injected noise does not increase with N , allowing coarse screening to retain more true top- k itemsets. Also, the estimation errors of DP-FM sketches reduce with N . Running time increases with N because more users need to be processed.

6.2.6 Impact of M . Figures 5(b) and 6(b) show the impact of number of attributes M in the dataset. PriVFim maintains stable F1 and NCR as M changes, while IE-Other and IE-Full first improve and then deteriorate as the number of attributes increases. This is because, as the number of attributes increases, IE-Other and IE-Full generate more DP-FM sketches for itemsets, making the estimation more accurate. However, a larger number of attributes requires IE-Other and IE-Full to use a fixed privacy budget to form more DP-FM sketches of complementary sets, thus introducing more noise. PriVFim avoids this by not adopting the intersection estimation method in Sec. 2.2, while using the MAP-based method.

6.2.7 Impact of $|I|$. Here we vary the size of the global item universe $|I|$. Note that varying $|I|$ also changes the domain size of each attribute s_i^j . Figures 5(c) and 6(c) show the results. On both datasets, the F1 and NCR of compared methods gradually increase. This is because a smaller $|I|$ results in more itemsets with the same large support count in the dataset. As $|I|$ increases, fewer itemsets have the same support count, so the interference among itemsets is reduced and the F1 and NCR scores increase accordingly.

6.2.8 Impact of first-round report size $|\mathcal{P}_i|$. $|\mathcal{P}_i|$ represents the number of selected items and their noisy support counts reported by data party i in the first interaction for coarse screening. By default, we set $|\mathcal{P}_i| = k$. Figure 7 evaluates different report sizes ranging from $0.5k$ to $2k$. The F1 and NCR of compared methods remain nearly unchanged as $|\mathcal{P}_i|$ increases, because additional reported items do not provide more candidate itemsets. The running time also changes little over the tested range. Therefore, the default setting $|\mathcal{P}_i| = k$ is sufficient for coarse screening.

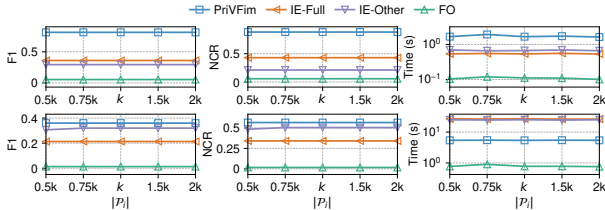


Figure 7: F1, NCR, and running time varying $|\mathcal{P}_i|$ on Default-Credit (first row) and MiniBooNE (second row) datasets.

6.2.9 Impact of second-round report size $|Q_i|$. In the second interaction, $|Q_i|$ controls the number of selected local itemsets and

corresponding DP-FM sketches reported by data party i for refined estimation. We set $|Q_i| = k$ by default in PriVFim and vary this value from $0.5k$ to $2k$. Figure 8 presents the results. The F1 and NCR values remain stable across different report sizes, indicating that the tested settings all preserve the necessary DP-FM sketches for estimating the top- k itemsets among the candidates. The running time also exhibits little variation. Thus, increasing or decreasing $|Q_i|$ provides limited benefit, and we retain the default setting $|Q_i| = k$.

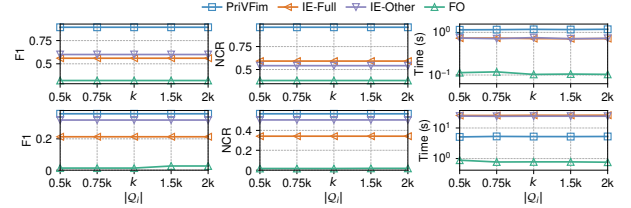


Figure 8: F1, NCR, and running time varying $|Q_i|$ on Bank (first row) and MiniBooNE (second row) datasets.

6.2.10 Impact of candidate set size $|C_{2k}|$. $|C_{2k}|$ represents the number of candidate itemsets retained after coarse screening and passed to MAP-based refined estimation. The default candidate set size is $2k$. Figure 9 studies the impact of it by varying the candidate size from k to $5k$. On Census, both F1 and NCR of PriVFim increase when the candidate size grows from k to $1.5k$, since enlarging C_{2k} allows more true top- k itemsets to be retained after coarse screening. However, further enlarging the candidate set brings limited improvement. On PokerHand, both F1 and NCR of PriVFim remain almost unchanged in the tested range. Thus, we choose $|C_{2k}| = 2k$, which provides sufficient candidate coverage while avoiding unnecessary computation from a large candidate set.

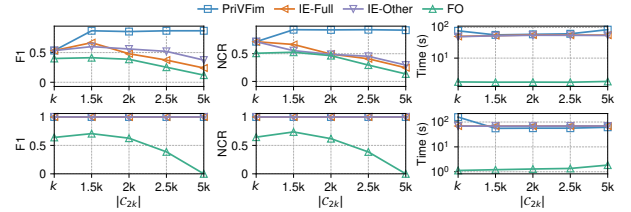


Figure 9: F1, NCR, and running time varying $|C_{2k}|$ on Census (first row) and PokerHand (second row) datasets.

6.2.11 Impact of budget partition. In PriVFim, we evenly allocated the privacy budget for the two rounds of data collection, i.e., setting $\epsilon_1 = \epsilon_2 = 0.5\epsilon$. To evaluate the impact of budget partition, we vary the first-round budget proportion ϵ_1/ϵ . Figure 10 shows the results. As ϵ_1/ϵ increases, PriVFim remains relatively stable, while the F1 and NCR of IE-Full and IE-Other gradually decrease. This is because, as ϵ_1/ϵ increases, the budget ϵ_2 is smaller, causing more noise to be added in the second round of IE-Full and IE-Other. PriVFim is less affected because its more advanced MAP-based estimation injects a small amount of noise even when the budget is limited. Besides, FO is also sensitive to the budget partition.

6.2.12 Comparing with one-round baselines. Figure 11 compares PriVFim with two baselines that adopt only one round of interaction. In *First-round*, each party only sends noisy item support counts as the first interaction of PriVFim and the server estimates

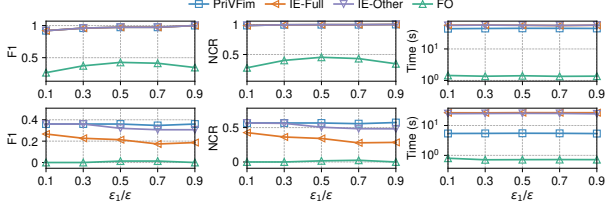


Figure 10: F1, NCR, and running time varying first-round budget proportion ϵ_1/ϵ on PokerHand (first row) and Mini-BooNE (second row) datasets.

the support counts with Eq. (2). In *Second-round*, each parties only sends DP-FM sketches as the second interaction of PrivFim and the server applies the MAP-based estimation method to derive the estimated support count. As shown, PrivFim outperforms both one-round baselines, proving the necessity of two-round interaction. On Bank, the mining accuracy of First-round is close to that of PrivFim since in this dataset, more top- k frequent itemsets are 1-itemsets, which can be easily selected according to the reports of First-round. In contrast, on DefaultCredit, top- k frequent itemsets contain more n -itemsets ($n > 1$). Thus, First-round achieves worse mining accuracy in this dataset while Second-round, which adopted the MAP-based estimation, outperforms First-round.

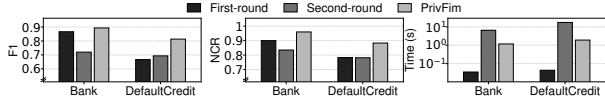


Figure 11: Comparison of one-round baselines and PrivFim.

6.2.13 Communication cost. Table 3 shows the communication costs of the compared methods. We randomly sample different subsets of datasets with different numbers of users N , different numbers of attributes M , and different sizes of global item universe $|I|$. PrivFim’s communication cost changes little with the number of users because the size of the FM sketches is fixed. IE-Full usually sends more data, especially when merged attributes have larger domains that require more DP-FM sketches for complementary sets. IE-Other reduces this cost by directly obtaining the complementary set of the set to be intersected. FO sometimes sends fewer bytes than PrivFim because it replaces sketches with OUE, but its F1 and NCR are much lower as in Figures 3 and 4.

Table 3: Communication costs (MiB) varying N , M and $|I|$ in Bank and DefaultCredit datasets.

Variable	Bank					DefaultCredit				
	Value	PrivFim	IE-Full	IE-Other	FO	Value	PrivFim	IE-Full	IE-Other	FO
N	20%	0.179	0.402	0.224	0.084	20%	0.320	1.210	0.305	0.061
	40%	0.179	0.396	0.224	0.161	40%	0.329	1.210	0.302	0.120
	60%	0.182	0.396	0.224	0.225	60%	0.345	1.210	0.302	0.178
	80%	0.182	0.396	0.224	0.291	80%	0.348	1.210	0.302	0.233
	100%	0.182	0.396	0.224	0.367	100%	0.348	1.210	0.302	0.288
M	4	0.238	16.77	0.279	0.373	5	0.247	197.9	0.276	0.273
	7	0.266	4.440	0.317	0.373	9	0.269	11.44	0.314	0.274
	11	0.222	1.420	0.286	0.365	14	0.216	1.803	0.255	0.282
	14	0.198	0.459	0.205	0.360	18	0.188	1.295	0.233	0.281
	18	0.182	0.396	0.224	0.367	23	0.348	1.210	0.302	0.288
I	31	0.238	0.182	0.354	0.222	43	0.228	0.214	0.261	0.210
	57	0.193	0.182	0.261	0.296	86	0.295	0.562	0.339	0.279
	87	0.182	0.271	0.249	0.350	143	0.288	0.828	0.330	0.274
	115	0.185	0.368	0.255	0.362	187	0.329	1.019	0.314	0.284
	144	0.182	0.396	0.224	0.367	229	0.348	1.210	0.302	0.288

7 RELATED WORK

DP for Item/Itemset Mining. Early work considered centralized data. Zeng *et al.* [67] analyzed the utility challenges of DP frequent itemset mining, while PrivBasis [34] and subsequent methods [9, 51, 56] reduced the number or sensitivity of support queries to get better utility. DP-Eclat [26] perturbed support counts over record-ID lists. The sample-and-threshold mechanism [18] supported sparse item discovery. These methods nevertheless assume that complete records are centrally available. Under local DP, Qin *et al.* [46] studied heavy hitters in set-valued data, while Wang *et al.* [60] proposed SVM and SVSM for item/itemset mining. Later work addressed item discovery over large or structured domains [55, 61], dedicated LDP itemset mining [17, 33], and adaptive top- k , streaming, and multi-class item mining [21, 36, 39]. However, these methods assume that each user has a complete record. In the federated setting, studies mainly target horizontal data, covering heavy-hitter discovery [13, 48, 70, 74] and itemset mining [16], with some combining DP and secure computation [7, 15]. However, these methods do not consider the VFL setting, where no party has a complete record and itemset support counts must be estimated from user-set intersections.

DP for Federated Learning. Most DP research for FL considers the horizontal setting, where parties hold different users with shared features. Building on DP-SGD [4], early studies adapted gradient clipping and noise injection for record- or client-level DP [25, 40]. Later work analyzed convergence under noisy aggregation [62], improved clipping strategies [6, 68], and addressed heterogeneous data [45]. Naseri *et al.* [43] further compared local and central DP in terms of privacy and robustness. Another line of work reduces trust in the server, where secure aggregation [5, 8, 30] and private optimization [38] were adopted. Beyond model training, DP has been applied to heavy-hitter discovery [13, 70, 74] and range queries [32]. DP research in VFL is less extensive. VAFL [14] protects intermediate representations in asynchronous training, while Xie *et al.* [64] study communication-efficient privacy-preserving VFL based on ADMM. OpBoost [35] studies vertical tree boosting. Li *et al.* [37] use private FM sketches for vertical clustering. VertiMRF [71] explored data synthesis under VFL. However, these methods do not address the candidate search and private intersection estimation required by VFL itemset mining, which are the focus of our work.

8 CONCLUSION

We conduct a comprehensive study on differentially private frequent itemset mining in VFL. We propose PrivFim, a coarse-to-fine, two-round interactive framework for differentially private frequent itemset mining under VFL. In the first interaction, data parties report their noisy local item counts and the server constructs an itemset-enumeration tree with a dynamic pruning strategy to reduce the search space and identify promising candidate itemsets. In the second interaction, PrivFim uses differentially private Flajolet-Martin sketches and a Maximum A Posteriori (MAP) method to estimate the support counts of cross-party candidate itemsets and select the final global top- k itemsets. Our theoretical analysis establishes the privacy and utility guarantees and the computational and communication costs of PrivFim. Extensive experiments show that PrivFim improves mining accuracy over baseline methods while maintaining low computational and communication overhead.

REFERENCES

- [1] 2016. General Data Protection Regulation. <https://gdpr-info.eu/>
- [2] 2018. California Consumer Privacy Act. <https://oag.ca.gov/privacy/ccpa>
- [3] 2026. PrivFim: Differentially Private Vertical Federated Frequent Itemset Mining (Technical Report). <https://anonymous.4open.science/r/PrivFim-F2B0/>.
- [4] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep Learning with Differential Privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 308–318.
- [5] Naman Agarwal, Peter Kairouz, and Ziyu Liu. 2021. The Skellam Mechanism for Differentially Private Federated Learning. In *Advances in Neural Information Processing Systems*, Vol. 34. 5052–5064.
- [6] Galen Andrew, Om Thakkar, H. Brendan McMahan, and Swaroop Ramaswamy. 2021. Differentially Private Learning with Adaptive Clipping. In *Advances in Neural Information Processing Systems*, Vol. 34. 17455–17466.
- [7] Jonas Böhler and Florian Kerschbaum. 2021. Secure multi-party computation of differentially private heavy hitters. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2361–2377.
- [8] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical Secure Aggregation for Privacy-Preserving Machine Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1175–1191.
- [9] Luca Bonomi and Li Xiong. 2013. Mining frequent patterns with differential privacy. *Proceedings of the VLDB Endowment* 6, 12 (2013), 1422–1427.
- [10] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks. In *28th USENIX Security Symposium*. USENIX Association, 267–284.
- [11] Robert Catral and Franz Oppacher. 2002. Poker Hand. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5KW38>.
- [12] Census-Income KDD 2000. Census-Income (KDD). UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5N30T>.
- [13] Karan Chadha, Junye Chen, John Duchi, Vitaly Feldman, Hanieh Hashemi, Omid Javidiakht, Audra McMillan, and Kunal Talwar. 2024. Differentially private heavy hitter detection using federated analytics. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 512–533.
- [14] Tianyi Chen, Xiao Jin, Yuejiao Sun, and Wotao Yin. 2020. VAFL: A Method of Vertical Asynchronous Federated Learning. *arXiv preprint arXiv:2007.06081* (2020).
- [15] Wenjie Chen, Haoyu Chen, Tingxuan Han, Wei Tong, and Sheng Zhong. 2024. Secure two-party frequent itemset mining with guaranteeing differential privacy. *IEEE Transactions on Mobile Computing* 24, 1 (2024), 276–292.
- [16] Yao Chen, Wensheng Gan, Yongdong Wu, and Philip S Yu. 2023. Privacy-preserving federated mining of frequent itemsets. *Information Sciences* 625 (2023), 504–520.
- [17] Zhili Chen and Jiali Wang. 2022. Ldp-fpminer: Fp-tree based frequent itemset mining with local differential privacy. *arXiv preprint arXiv:2209.01333* (2022).
- [18] Graham Cormode and Akash Bharadwaj. 2022. Sample-and-threshold differential privacy: Histograms and applications. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1420–1431.
- [19] Default of Credit Card Clients 2009. Default of Credit Card Clients. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/350/default+of+credit+card-clients>
- [20] Linkang Du, Zhikun Zhang, Shaojie Bai, Changchang Liu, Shouling Ji, Peng Cheng, and Jiming Chen. 2021. AHEAD: Adaptive Hierarchical Decomposition for Range Query under Local Differential Privacy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 1266–1288.
- [21] Rong Du, Qingqing Ye, Yue Fu, Haibo Hu, and Kai Huang. 2024. Top-k discovery under local differential privacy: An adaptive sampling approach. *IEEE Transactions on Dependable and Secure Computing* 22, 2 (2024), 1763–1780.
- [22] John C. Duchi, Michael I. Jordan, and Martin J. Wainwright. 2018. Minimax Optimal Procedures for Locally Private Estimation. *J. Amer. Statist. Assoc.* 113, 521 (2018), 182–201.
- [23] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.
- [24] Úlfar Erlingsson, Vasily Pihur, and Aleksandra Korolova. 2014. RAPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 1054–1067.
- [25] Robin C. Geyer, Tassilo Klein, and Moin Nabi. 2017. Differentially Private Federated Learning: A Client Level Perspective. *arXiv preprint arXiv:1712.07557* (2017).
- [26] Shengyi Guan, Xuebin Ma, Wuyungerli Li, and Xiangyu Bai. 2020. DP-Eclat: a vertical frequent itemset mining algorithm based on differential privacy. In *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 1696–1703.
- [27] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Wei Ni. 2025. Robust Privacy-Preserving Triangle Counting under Edge Local Differential Privacy. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.
- [28] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally Differentially Private Analysis of Graph Statistics. In *30th USENIX Security Symposium*. USENIX Association, 983–1000.
- [29] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Communication-Efficient Triangle Counting under Local Differential Privacy. In *31st USENIX Security Symposium*. USENIX Association, 537–554.
- [30] Peter Kairouz, Ziyu Liu, and Thomas Steinke. 2021. The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation. In *International Conference on Machine Learning*. PMLR, 5201–5212.
- [31] Georgios Kellaris, Stavros Papadopoulos, Xiaokui Xiao, and Dimitris Papadias. 2014. Differentially Private Event Sequences over Infinite Streams. *Proceedings of the VLDB Endowment* 7, 12 (2014), 1155–1166.
- [32] Ala Eddine Laouir and Abdessamad Imine. 2025. Private Approximate Query over Horizontal Data Federation. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT) (Advances in Database Technology)*, Vol. 28. OpenProceedings.org. <https://doi.org/10.48786/edbt.2025.11>
- [33] Junhui Li, Wensheng Gan, Yijie Gui, Yongdong Wu, and Philip S Yu. 2022. Frequent itemset mining with local differential privacy. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 1146–1155.
- [34] Ninghui Li, Wahbeh Qardaji, Dong Su, and Jianneng Cao. 2012. Privbasis: Frequent itemset mining with differential privacy. *arXiv preprint arXiv:1208.0093* (2012).
- [35] Qibin Li, Zeyi Wen, and Bingsheng He. 2022. OpBoost: A Vertical Federated Tree Boosting Framework Based on Order-Preserving Desensitization. *Proceedings of the VLDB Endowment* 16, 2 (2022), 202–215.
- [36] Xiaochen Li, Weiran Liu, Jian Lou, Yuan Hong, Lei Zhang, Zhan Qin, and Kui Ren. 2024. Local differentially private heavy hitter detection in data streams with bounded memory. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27.
- [37] Zitao Li, Tianhao Wang, and Ninghui Li. 2023. Differentially Private Vertical Federated Clustering. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1234–1247.
- [38] Andrew Lowy and Meisam Razaviyayn. 2023. Private Federated Learning Without a Trusted Server: Optimal Algorithms for Convex Losses. In *International Conference on Learning Representations*.
- [39] Yulian Mao, Qingqing Ye, Rong Du, Qi Wang, Kai Huang, and Haibo Hu. 2025. Multi-class item mining under local differential privacy. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3549–3561.
- [40] H. Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. 2018. Learning Differentially Private Recurrent Language Models. In *International Conference on Learning Representations*.
- [41] MiniBooNE 2005. MiniBooNE Particle Identification. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/199/miniboone+particle+identification>
- [42] S. Moro, P. Rita, and P. Cortez. 2014. Bank Marketing. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5K306>.
- [43] Mohammad Naseri, Jamie Hayes, and Emiliano De Cristofaro. 2022. Local and Central Differential Privacy for Robustness and Privacy in Federated Learning. In *Network and Distributed System Security Symposium*.
- [44] Milad Nasr, Reza Shokri, and Amir Houmansadr. 2019. Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-Box Inference Attacks against Centralized and Federated Learning. In *2019 IEEE Symposium on Security and Privacy*. IEEE, 739–753.
- [45] Maxime Noble, Aurélien Bellet, and Aymeric Dieuleveut. 2022. Differentially Private Federated Learning on Heterogeneous Data. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 10110–10145.
- [46] Zhan Qin, Yin Yang, Ting Yu, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2016. Heavy hitter estimation over set-valued data with local differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 192–203.
- [47] Xuebin Ren, Liang Shi, Weiren Yu, Shusen Yang, Cong Zhao, and Zongben Xu. 2022. LDP-IDS: Local Differential Privacy for Infinite Data Streams. In *Proceedings of the 2022 International Conference on Management of Data*. 1064–1077.
- [48] Jiaqi Shao, Shanshan Han, Chaoyang He, and Bing Luo. 2023. Privacy-preserving federated heavy hitter analytics for non-iiid data. *ICML WS* (2023).
- [49] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership Inference Attacks Against Machine Learning Models. In *2017 IEEE Symposium on Security and Privacy*. IEEE, 3–18.
- [50] Adam Smith, Shuang Song, and Abhradeep Guha Thakurta. 2020. The flajolet-martin sketch itself preserves differential privacy: Private counting with minimal space. *Advances in Neural Information Processing Systems* 33 (2020), 19561–19572.
- [51] Sen Su, Shengzhi Xu, Xiang Cheng, Zhengyi Li, and Fangchun Yang. 2015. Differentially private frequent itemset mining via transaction splitting. *IEEE transactions on knowledge and data engineering* 27, 7 (2015), 1875–1891.

- [52] Jaideep Vaidya and Chris Clifton. 2002. Privacy preserving association rule mining in vertically partitioned data. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 639–644.
- [53] Leixia Wang, Qingqing Ye, Haibo Hu, and Xiaofeng Meng. 2024. PriPL-Tree: Accurate Range Query for Arbitrary Distribution under Local Differential Privacy. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3031–3044.
- [54] Ning Wang, Yaohua Wang, Zhigang Wang, Jie Nie, Zhiqiang Wei, Peng Tang, Yu Gu, and Ge Yu. 2023. PrivNUD: Effective Range Query Processing under Local Differential Privacy. In *2023 IEEE 39th International Conference on Data Engineering*. IEEE, 2660–2672.
- [55] Ning Wang, Xiaokui Xiao, Yin Yang, Ta Duy Hoang, Hyejin Shin, Junbum Shin, and Ge Yu. 2018. PrivTrie: Effective frequent term discovery under local differential privacy. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 821–832.
- [56] Ning Wang, Xiaokui Xiao, Yin Yang, Zhenjie Zhang, Yu Gu, and Ge Yu. 2017. PrivSuper: A superset-first approach to frequent itemset mining under differential privacy. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 809–820.
- [57] Ning Wang, Xiaokui Xiao, Yin Yang, Jun Zhao, Siu Cheung Hui, Hyejin Shin, Junbum Shin, and Ge Yu. 2019. Collecting and Analyzing Multidimensional Data with Local Differential Privacy. In *2019 IEEE 35th International Conference on Data Engineering*. IEEE, 638–649.
- [58] Tianhao Wang, Jeremiah Blocki, Ninghui Li, and Somesh Jha. 2017. Locally Differentially Private Protocols for Frequency Estimation. In *26th USENIX Security Symposium*. USENIX Association, 729–745.
- [59] Tianhao Wang, Jun-Qi Chen, Zhikun Zhang, Dong Su, Yueqiang Cheng, Zhou Li, Ninghui Li, and Somesh Jha. 2021. Continuous Release of Data Streams under Both Centralized and Local Differential Privacy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 1237–1253.
- [60] Tianhao Wang, Ninghui Li, and Somesh Jha. 2018. Locally differentially private frequent itemset mining. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 127–143.
- [61] Tianhao Wang, Ninghui Li, and Somesh Jha. 2019. Locally differentially private heavy hitter identification. *IEEE Transactions on Dependable and Secure Computing* 18, 2 (2019), 982–993.
- [62] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H. Yang, Farhad Farokhi, Shi Jin, Tony Q. S. Quek, and H. Vincent Poor. 2020. Federated Learning with Differential Privacy: Algorithms and Performance Analysis. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3454–3469.
- [63] Yuncheng Wu, Shaofeng Cai, Xiaokui Xiao, Gang Chen, and Beng Chin Ooi. 2020. Privacy Preserving Vertical Federated Learning for Tree-Based Models. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2090–2103. <https://doi.org/10.14778/3407790.3407811>
- [64] Chulin Xie, Pin-Yu Chen, Qimbin Li, Arash Nourian, Ce Zhang, and Bo Li. 2024. Improving Privacy-Preserving Vertical Federated Learning by Efficient Communication with ADMM. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning*. IEEE, 443–471.
- [65] Jianyu Yang, Tianhao Wang, Ninghui Li, Xiang Cheng, and Sen Su. 2020. Answering Multi-Dimensional Range Queries under Local Differential Privacy. *Proceedings of the VLDB Endowment* 14, 3 (2020), 378–390.
- [66] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* 10, 2 (2019), 1–19.
- [67] Chen Zeng, Jeffrey F Naughton, and Jin-Yi Cai. 2012. On differentially private frequent itemset mining. *The VLDB journal: very large data bases: a publication of the VLDB Endowment* 6, 1 (2012), 25.
- [68] Xinwei Zhang, Xiangyi Chen, Mingyi Hong, Zhiwei Steven Wu, and Jinfeng Yi. 2022. Understanding Clipping for Federated Learning: Convergence and Client-Level Differential Privacy. In *International Conference on Machine Learning*. PMLR, 26048–26067.
- [69] Yuheng Zhang, Ruoxi Jia, Hengzhi Pei, Wenxiao Wang, Bo Li, and Dawn Song. 2020. The Secret Revealer: Generative Model-Inversion Attacks Against Deep Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 253–261.
- [70] Yuemin Zhang, Qingqing Ye, and Haibo Hu. 2025. Federated heavy hitter analytics with local differential privacy. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.
- [71] Fangyuan Zhao, Zitao Li, Xuebin Ren, Bolin Ding, Shusen Yang, and Yaliang Li. 2024. Vertimrf: Differentially private vertical federated data synthesis. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 4431–4442.
- [72] Zhen Zhao, Lei Lan, Baocang Wang, and Jianchang Lai. 2023. Verifiable Privacy-Preserving Outsourced Frequent Itemset Mining on Vertically Partitioned Databases. *Electronics* 12, 8 (2023), 1952.
- [73] Peijia Zheng, Ziyang Cheng, Xianhao Tian, Hongmei Liu, Weiqi Luo, and Jiwu Huang. 2023. Non-interactive privacy-preserving frequent itemset mining over encrypted cloud data. *IEEE Transactions on Cloud Computing* 11, 4 (2023), 3452–3468.
- [74] Wennan Zhu, Peter Kairouz, Brendan McMahan, Haicheng Sun, and Wei Li. 2020. Federated heavy hitters discovery with differential privacy. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3837–3847.

A PROOFS OF THEOREMS

A.1 Proof of Theorem 4.1

Let $A_i^{p,g}$ denote the random output of the g -th DP-FM repetition, and let $\alpha_i^{p,g}$ denote its observed value. Conditional on $n_i^p = n$, let $Z_1, \dots, Z_{n+n_p}$ be the hash values of the n real elements and the n_p phantom elements used in this repetition. Under the ideal geometric-value hash assumption, these values are independent and have the same CDF F . By the DP-FM construction,

$$A_i^{p,g} = \max\{\alpha_{\min}, Z_1, \dots, Z_{n+n_p}\}. \quad (7)$$

Consider an integer threshold t . If $t < \alpha_{\min}$, the output cannot be at most t . Hence,

$$\Pr(A_i^{p,g} \leq t \mid n_i^p = n) = 0. \quad (8)$$

Now suppose that $t \geq \alpha_{\min}$. In this case, $A_i^{p,g} \leq t$ if and only if all real and phantom hash values are at most t . Therefore, independence gives

$$\Pr(A_i^{p,g} \leq t \mid n_i^p = n) = \Pr(Z_1 \leq t, \dots, Z_{n+n_p} \leq t) \quad (9)$$

$$= \prod_{\ell=1}^{n+n_p} \Pr(Z_\ell \leq t) \quad (10)$$

$$= F(t)^{n+n_p}. \quad (11)$$

Combining the two cases proves the stated expression for $F_i^p(t \mid n)$.

Finally, $A_i^{p,g}$ is integer-valued. Thus, for every integer a ,

$$P_i^p(a \mid n) = \Pr(A_i^{p,g} = a \mid n_i^p = n) \quad (12)$$

$$= \Pr(A_i^{p,g} \leq a \mid n_i^p = n) - \Pr(A_i^{p,g} \leq a-1 \mid n_i^p = n) \quad (13)$$

$$= F_i^p(a \mid n) - F_i^p(a-1 \mid n). \quad (14)$$

This completes the proof.

A.2 Proof of Theorem 4.3

Let $\mathcal{U}$ be the aligned user universe, where $|\mathcal{U}| = N$, and let $m = |\mathcal{B}^{(r)}| \geq 1$. By the definition of the support count,

$$C^{(r)} = \left| \bigcap_{(i,p) \in \mathcal{B}^{(r)}} \mathcal{X}_i^p \right|. \quad (15)$$

The intersection is a subset of every participating local user set. Consequently, for every $(i,p) \in \mathcal{B}^{(r)}$,

$$C^{(r)} \leq |\mathcal{X}_i^p| = n_i^p, \quad (16)$$

and hence

$$C^{(r)} \leq \min_{(i,p) \in \mathcal{B}^{(r)}} n_i^p. \quad (17)$$

For the lower bound, De Morgan's law and the union bound for set cardinalities yield

$$C^{(r)} = N - \left| \mathcal{U} \setminus \bigcap_{(i,p) \in \mathcal{B}^{(r)}} \mathcal{X}_i^p \right| \quad (18)$$

$$= N - \left| \bigcup_{(i,p) \in \mathcal{B}^{(r)}} (\mathcal{U} \setminus \mathcal{X}_i^p) \right| \quad (19)$$

$$\geq N - \sum_{(i,p) \in \mathcal{B}^{(r)}} |\mathcal{U} \setminus \mathcal{X}_i^p| \quad (20)$$

$$= N - \sum_{(i,p) \in \mathcal{B}^{(r)}} (N - n_i^p) \quad (21)$$

$$= \sum_{(i,p) \in \mathcal{B}^{(r)}} n_i^p - (m-1)N. \quad (22)$$

Since a cardinality is nonnegative, it follows that

$$C^{(r)} \geq \max \left\{ 0, \sum_{(i,p) \in \mathcal{B}^{(r)}} n_i^p - (|\mathcal{B}^{(r)}| - 1)N \right\}. \quad (23)$$

Together with the upper bound, this proves the theorem.

A.3 Proof of Theorem 4.5

Fix a repetition index g and a candidate intersection size $n \in \mathcal{N}^{(r)}$. Write $\mathcal{B} = \mathcal{B}^{(r)}$ for brevity, and let $A_i^{p,g}$ denote the random DP-FM output whose observed value is $\alpha_i^{p,g}$. By the definition of $\mathcal{N}^{(r)}$, we have $0 \leq n \leq \hat{n}_i^p$ for every $(i,p) \in \mathcal{B}$.

Let

$$\mathcal{X}_\cap^{(r)} = \bigcap_{(i,p) \in \mathcal{B}} \mathcal{X}_i^p.$$

The approximation condition $\mathcal{E}_{\text{disjoint}}$ introduced before the theorem assumes that, after removing $\mathcal{X}_\cap^{(r)}$, the remaining parts of different local user sets do not overlap. Formally,

$$\begin{aligned} (\mathcal{X}_i^p \setminus \mathcal{X}_\cap^{(r)}) \cap (\mathcal{X}_{i'}^{p'} \setminus \mathcal{X}_\cap^{(r)}) &= \emptyset, \\ \forall (i,p), (i',p') \in \mathcal{B} \text{ with } (i,p) &\neq (i',p'). \end{aligned}$$

Thus, the n users in $\mathcal{X}_\cap^{(r)}$ are the only users shared by different relevant sketches. Each sketch (i,p) contains these n common users and $\hat{n}_i^p - n$ other users. The latter users appear only in that sketch under $\mathcal{E}_{\text{disjoint}}$. All relevant sketches use the same DP-FM parameters, so they have the same n_p and $\alpha_{\min}$. Hash values of distinct real users are independent under the ideal hash assumption, and phantom hash values are generated independently of one another and of the real-user hash values. In the likelihood calculation, the local cardinalities are fixed at their estimates $\hat{n}_i^p$.

We first calculate the joint CDF of the outputs. For arbitrary integer thresholds $\{s_{i,p}\}_{(i,p) \in \mathcal{B}}$, let

$$s_{\min} = \min_{(i,p) \in \mathcal{B}} s_{i,p},$$

and define

$$\begin{aligned} G_g((s_{i,p})_{(i,p) \in \mathcal{B}} \mid n) \\ := \Pr(A_i^{p,g} \leq s_{i,p}, \forall (i,p) \in \mathcal{B} \mid C^{(r)} = n, \{n_i^p = \hat{n}_i^p\}_{(i,p) \in \mathcal{B}}, \mathcal{E}_{\text{disjoint}}). \end{aligned}$$

If $s_{\min} < \alpha_{\min}$, at least one output is required to be smaller than the lower bound $\alpha_{\min}$, so the joint probability is zero.

Now suppose that $s_{\min} \geq \alpha_{\min}$. All parties use the same hash function $H_{\mathcal{L}_g}$ in repetition g . Therefore, a common user has the same hash value in every relevant sketch. For all outputs to satisfy their respective thresholds, the hash value of each common user must not exceed $s_{\min}$. Since there are n common users, their probability contribution is $F(s_{\min})^n$.

For each $(i, p) \in \mathcal{B}$, the $\hat{n}_i^p - n$ other real-user hash values and the n_p phantom hash values must all be at most $s_{i,p}$. Their probability contribution is $F(s_{i,p})^{\hat{n}_i^p - n + n_p}$. Multiplying the contributions of the common users and all individual sketches gives

$$G_g((s_{i,p})_{(i,p) \in \mathcal{B}} \mid n) = \begin{cases} 0, & s_{\min} < \alpha_{\min}, \\ F(s_{\min})^n \prod_{(i,p) \in \mathcal{B}} F(s_{i,p})^{\hat{n}_i^p - n + n_p}, & s_{\min} \geq \alpha_{\min}. \end{cases}$$

We next obtain the probability of observing the exact output values. For a single integer-valued random variable, the probability of an exact value equals the difference between its CDF at that value and at the preceding integer. Applying this difference to every output and expanding the resulting product gives

$$\mathcal{L}_g^{(r)}(n) = \sum_{W \subseteq \mathcal{B}} (-1)^{|W|} G_g((t_{i,p}^{g,W})_{(i,p) \in \mathcal{B}} \mid n),$$

where $t_{i,p}^{g,W} = \alpha_i^{p,g} - \mathbb{I}((i,p) \in W)$. For this threshold vector, the minimum threshold is $t_{\min}^{g,W}$. Substituting the joint CDF derived above therefore gives

$$G_g((t_{i,p}^{g,W})_{(i,p) \in \mathcal{B}} \mid n) = L(W, n).$$

Consequently,

$$\mathcal{L}_g^{(r)}(n) = \sum_{W \subseteq \mathcal{B}^{(r)}} (-1)^{|W|} L(W, n),$$

which is the claimed result. The zero case in $L(W, n)$ also covers an observation equal to $\alpha_{\min}$: if a term replaces this observation by $\alpha_{\min} - 1$, the corresponding joint CDF is zero.

The expression above is exact when the local cardinalities are fixed at $\hat{n}_i^p$ and $\mathcal{E}_{\text{disjoint}}$ is assumed. It is used as an approximate likelihood for the actual outputs because $\hat{n}_i^p$ are estimates and the unknown partial overlaps outside $\mathcal{X}_{\cap}^{(r)}$ are not included. This completes the proof.

A.4 Proof of Theorem 5.1

Consider two neighboring inputs that differ in one user's local record. In the first interaction, the Laplace mechanisms for the released counts use a total privacy budget of ϵ_1 . By sequential composition, all reports released in this interaction satisfy ϵ_1 -DP. Selecting the local top- k items and constructing the candidate item-sets use only these private reports and therefore cause no additional privacy loss.

In the second interaction, each party releases at most k DP-FM sketch vectors. Each vector is assigned $(\epsilon_2/k, \delta/k)$. Since $\epsilon_2/k \leq 2 \log(k/\delta) = 2 \log(1/(\delta/k))$, Lemma 2.2 shows that each vector satisfies $(\epsilon_2/k, \delta/k)$ -DP when its single-run privacy parameter is

set as specified in that lemma. Sequential composition over at most k vectors gives (ϵ_2, δ) -DP for the second interaction.

For a neighboring pair, only one party's local input changes. Conditional on the first-interaction transcript, the reports of the other parties depend on the changed record only through that transcript and are therefore post-processing with respect to this record. Sequential composition of the two interactions gives $(\epsilon_1 + \epsilon_2, \delta) = (\epsilon, \delta)$ -DP. MAP estimation, sorting, and final top- k selection are also post-processing of the private reports and add no privacy loss. Hence, PriVFim satisfies (ϵ, δ) -DP.

A.5 Proof of Theorem 5.2

Using the notation in the theorem, the likelihood of a possible cardinality $n \in \mathcal{S}$ over all h repetitions is $\prod_{g=1}^h \ell_g(n)$. Since the prior on $\mathcal{S}$ is uniform, the MAP estimate c_{MAP} maximizes this likelihood, or equivalently $\sum_{g=1}^h \log \ell_g(n)$.

Fix an incorrect value $n \in \mathcal{S}$ such that $|n - c| \geq q$. If its likelihood is at least that of c , then $\sum_{g=1}^h Z_{n,g} \leq 0$. The variables $Z_{n,1}, \dots, Z_{n,h}$ are independent, take values in $[-B, B]$, and have expectations of at least Δ . Hoeffding's inequality gives

$$\Pr\left(\sum_{g=1}^h Z_{n,g} \leq 0\right) \leq \exp\left(-\frac{h\Delta^2}{2B^2}\right). \quad (24)$$

The event $|c_{\text{MAP}} - c| \geq q$ implies that at least one $n \in \mathcal{S}$ satisfying $|n - c| \geq q$ has a likelihood no smaller than that of c . Applying the union bound over at most $|\mathcal{S}|$ such values yields $\Pr(|c_{\text{MAP}} - c| \geq q) \leq |\mathcal{S}| \exp(-h\Delta^2/(2B^2))$.

A.6 Proof of Theorem 5.3

Let $\mathcal{E}_L$ be the event that every reported local cardinality is estimated correctly. Since cardinalities are integers, an incorrect local estimate satisfies $|\hat{n}_i^p - n_i^p| \geq 1$. Theorem 5.2 with $q = 1$ gives an error probability of at most $(\hat{N}+1) \exp(-h\Delta^2/(2B^2))$ for each local sketch. There are at most Kk reported local sketches, so the union bound gives

$$\Pr(\mathcal{E}_L^c) \leq Kk(\hat{N}+1) \exp\left(-\frac{h\Delta^2}{2B^2}\right). \quad (25)$$

For each candidate involving multiple local sets, consider the MAP estimate that uses the true local cardinalities and denote it by $C_{\text{or}}^{(r)}$. By Theorem 5.2 with $q = \Gamma/2$ and $|\mathcal{N}^{(r)}| \leq \hat{N} + 1$, the probability that $|C_{\text{or}}^{(r)} - C^{(r)}| \geq \Gamma/2$ is at most $(\hat{N} + 1) \exp(-h\Delta^2/(2B^2))$. There are at most $2k$ such candidates. Hence, if $\mathcal{E}_I$ denotes the event that all these estimates have errors smaller than $\Gamma/2$, then

$$\Pr(\mathcal{E}_I^c) \leq 2k(\hat{N} + 1) \exp\left(-\frac{h\Delta^2}{2B^2}\right). \quad (26)$$

On $\mathcal{E}_L$, the estimated local cardinalities equal the true ones. Therefore, the actual intersection estimate $\hat{C}^{(r)}$ equals $C_{\text{or}}^{(r)}$ for every candidate involving multiple local sets. A candidate involving only one local set also has its exact support count on $\mathcal{E}_L$. Thus, on $\mathcal{E}_L \cap \mathcal{E}_I$, every candidate $X \in \mathcal{J}_{2k}$ satisfies $|\hat{C}_X - C_X| < \Gamma/2$.

For any $X \in \mathcal{T}_k$ and $Y \in \mathcal{J}_{2k} \setminus \mathcal{T}_k$, the definition of Γ then gives

$$\hat{C}_X > C_X - \frac{\Gamma}{2} \geq \min_{Z \in \mathcal{T}_k} C_Z - \frac{\Gamma}{2} = \max_{Z \in \mathcal{J}_{2k} \setminus \mathcal{T}_k} C_Z + \frac{\Gamma}{2} > \hat{C}_Y. \quad (27)$$

Every true top- k itemset is therefore ranked above every other candidate. Since $\mathcal{T}_k \subseteq \mathcal{J}_{2k}$, PriVFim returns $\mathcal{T}_k$. Finally, the union bound gives

$$\Pr(\mathcal{E}_L \cap \mathcal{E}_I) \geq 1 - (K + 2)k(\hat{N} + 1) \exp\left(-\frac{h\Delta^2}{2B^2}\right), \quad (28)$$

which proves the result. $\square$

B DATASET DETAILS

We provide additional details of the five datasets used in our experiments. Each record is treated as one user, and each attribute value is encoded as an item. For each dataset, we report the number of users N , the number of attributes M , the total number of distinct items $|I|$, and the domain sizes of all attributes after preprocessing.

Bank. Bank is derived from telephone marketing campaigns of a Portuguese banking institution. After preprocessing, the dataset contains $N = 41,176$ users and $M = 18$ attributes. The attribute domain sizes range from 2 to 26, yielding $|I| = 144$ distinct items. In attribute order, the domain sizes are

$$\{7, 12, 4, 8, 3, 3, 3, 2, 10, 5, 7, 8, 3, 4, 26, 26, 11, 2\}.$$

Census. Census contains demographic and employment information from the 1994 and 1995 Current Population Surveys. After preprocessing, it contains $N = 299,285$ users and $M = 40$ attributes. The domain sizes range from 2 to 52, with $|I| = 567$ distinct items. In attribute order, they are

$$\{10, 9, 52, 47, 17, 11, 3, 7, 24, 15, 5, 10, 2, 3, 6, 8, 11, 11, 11, 6, \\ 6, 51, 38, 8, 10, 9, 10, 3, 4, 6, 5, 43, 43, 43, 5, 3, 3, 3, 4, 2\}.$$

DefaultCredit. DefaultCredit contains credit card client data from Taiwan, including credit limits, demographic information, repayment history, bill statements, and previous payments. After preprocessing, it contains $N = 30,000$ users and $M = 23$ attributes. The domain sizes range from 2 to 11, giving $|I| = 229$ distinct items:

$$\{10, 2, 7, 4, 10, 11, 11, 11, 11, 10, 10, 11, \\ 11, 11, 11, 11, 11, 11, 11, 11, 11, 11\}.$$

MiniBooNE. MiniBooNE contains particle measurements from the MiniBooNE experiment. It has $N = 130,064$ users and $M = 50$ attributes after preprocessing. The domain sizes range from 5 to 10, resulting in $|I| = 494$ distinct items. In attribute order, the domain sizes are

$$\{10, 10, 10, 10, 9, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, \\ 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, \\ 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 5, 10, 10, 10, 10\}.$$

PokerHand. PokerHand represents poker hands drawn from a standard 52-card deck. The dataset contains $N = 1,025,010$ users and $M = 10$ attributes. The five suit attributes each have 4 possible values, while the five rank attributes each have 13 possible values, giving $|I| = 85$ distinct items. Their domain sizes are

$$\{4, 13, 4, 13, 4, 13, 4, 13, 4, 13\}$$